

PICE: A Semantic-Driven Progressive Inference System for LLM Serving in Cloud-Edge Networks

Huiyou Zhan, Xuan Zhang, Haisheng Tan, *IEEE Senior Member*, Han Tian, Dongping Yong,
Junyang Zhang, and Xiang-Yang Li, *IEEE/ACM Fellow*

Abstract—Large language models (LLMs), while driving a new wave of interactive AI applications across numerous domains, suffer from high inference costs and heavy cloud dependency. Motivated by the redundancy phenomenon in linguistics, we propose a progressive inference paradigm over cloud and edge, i.e., firstly generating the sketch of the answer by LLMs at cloud, and then conducting parallel extension to fill in details by small models (SLMs) at edge. Progressive inference offers potential benefits to improve throughput and reduce inference latency while facing key implementation challenges, including decreased response quality from SLMs, a tradeoff between the brevity and comprehensiveness of sketches, as well as increased latency caused by network transmission and edge inference. In this work, we propose and implement PICE, an LLM serving system with semantic-level cloud-edge collaboration, enhancing inference throughput and quality through dynamic inference task scheduling, ensemble learning, and parallel edge inference. Extensive testbed experiments illustrate that our approach achieves $1.5\text{--}2\times$ throughput enhancement and up to 43% latency reduction, while also potentially enhancing the quality compared to SOTA systems.

I. INTRODUCTION

Large-scale language models (LLMs) such as GPT4, LLaMA, and Qwen [1]–[3] have exhibited unprecedented performance in natural language processing and chatbot systems. However, the exceptional capabilities of LLMs come at a significant inference cost, taking 22 seconds for Claude [4] (accessed through Slack API) and 43 seconds for Vicuna-33B [5] (running locally on one NVIDIA A100 GPU) to generate a response [6]. To efficiently serve LLMs has emerged as a crucial demand for LLM-based AI applications to provide an engaging user experience, as their interactive nature mandates low response latency and high throughput during inference.

Moreover, due to the massive size of LLMs, current systems heavily rely on cloud computing (Fig. 1(a) - left), suffering from heavy request loads, high operational costs, and privacy concerns [7]. To mitigate these issues, there has been a growing interest in designing smaller, cost-effective LLMs that can take advantage of the increasing availability of numerous and diverse edge computing devices. For instance, Meta provides multiple versions of Llama [2] with only 7 billion parameters small enough to run on edge devices (Fig. 1(a) - middle). Although this approach can enhance efficiency, it typically

compromises response quality. To balance the need for fast and cost-effective inference, a hybrid inference strategy has been proposed, utilizing a router to dynamically allocate queries to either a small model on the edge or a large model on the cloud based on the predicted query difficulty [8]–[10] (Fig. 1(a) - right). However, this coarse-grained scheduling method is overly reliant on the performance of the router, highlighting the need for a more generalized approach that effectively harmonizes inference efficiency with response quality.

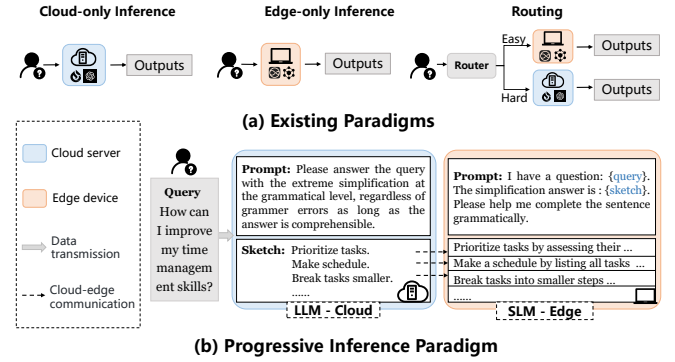


Fig. 1: Existing inference paradigms vs. progressive inference.

In contrast to existing efforts on optimizing inference at the model and system levels, we take a “semantic-level” pathway to improve throughput by reducing the output of LLMs, i.e., the LLM creates the response sketch and the SLM fills in the details. This perspective stems from how humans answer questions, i.e., we can convey the general meaning of a sentence with just a few words.

Motivated by this idea, we propose a progressive inference paradigm over cloud and edge (Fig. 1(b)). This architecture leverages the redundant computing power on the edge to offload workloads from the cloud while ensuring high response quality. Specifically, upon receiving a query, the LLM first evaluates the potential length of the response. If the response is concise and short, LLM will directly output a complete answer. In cases where a lengthy response is anticipated, LLM will produce semantically complete yet minimally grammatical sketches and then SLMs on the edge perform extension to improve inference efficiency. The semantic completeness of each short sentence eliminates sequential dependencies for their expansions, enabling SLMs to process each sentence in parallel and thereby breaking the sequential decoding of autoregressive models.

The progressive inference paradigm is promising while introducing several key challenges in practical implementation:

Huiyou Zhan, Xuan Zhang, Haisheng Tan, Dongping Yong, Junyang Zhang and Xiang-Yang Li are with the LINKE Lab and the CAS Key Laboratory of Wireless-Optical Communications, University of Science and Technology of China (USTC), Hefei 230027, China (e-mail: zhanhuiyou@foxmail.com; xuanzhang@mail.ustc.edu.cn; hstan@ustc.edu.cn; yongdongping@mail.ustc.edu.cn; zhangjunyang@mail.ustc.edu.cn; xiangyangli@ustc.edu.cn).

Han Tian is with the University of Science and Technology of China (USTC), Hefei 230027, China (e-mail: henrytian@ustc.edu.cn).

PICE: 一个语义驱动的渐进推理系统

用于云边网络中的大型语言模型服务

会友湛, 轩璋, 海生谭, *IEEE Senior Member*, 韩天, 东平永, 君阳张, 和向阳李,
IEEE/ACM Fellow

Abstract——大型语言模型（LLMs）在推动各个领域新一波交互式人工智能应用的同时，也存在高推理成本和严重依赖云端的问题。受到语言学中冗余现象的启发，我们提出了一种基于云端和边缘的渐进式推理范式，即首先在云端由LLMs生成答案草稿，然后在边缘由小模型（SLMs）进行并行扩展以填充细节。渐进式推理有望在提高吞吐量和降低推理延迟方面带来潜在优势，同时也面临关键的实现挑战，包括SLMs导致的响应质量下降、草稿简洁性与全面性之间的权衡，以及网络传输和边缘推理造成的延迟增加。在本工作中，我们提出并实现了PICE，一种具有语义级云-边协作的LLM服务系统，通过动态推理任务调度和集成学习提高推理吞吐量和质量。

会影响响应质量。为了在快速且成本效益高的推理需求之间取得平衡，提出了一种混合推理策略，利用一个路由器根据预测的查询难度，将查询动态分配到边缘的小模型或云端的大模型 [8]–[10]（图 1(a) - 右）。然而，这种粗粒度的调度方法过度依赖路由器的性能，因此需要一种更通用的方法，以有效地协调推理效率和响应质量。

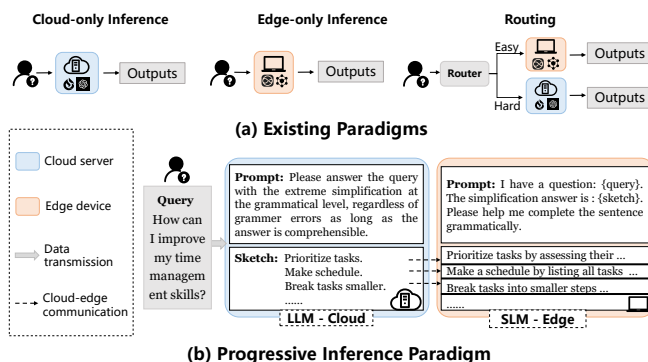


图1: 现有推理范式与渐进式推理。

一、引言

大型语言模型（LLMs），如 GPT4、LLaMA 和 Qwen [1]–[3]，在自然语言处理和聊天机器人系统中展现了前所未有的性能。然而，LLMs 的卓越能力伴随着显著的推理成本，例如 Claude [4]（通过 Slack API 访问）生成响应需要 22 秒，而 Vicuna-33B [5]（在一块 NVIDIA A100 GPU 上本地运行）生成响应需要 43 秒 [6]。如何高效地服务 LLMs 已成为基于 LLM 的 AI 应用提供良好用户体验的关键需求，因为它们的交互性质要求在推理过程中具有低响应延迟和高吞吐量。

此外，由于大型语言模型（LLM）的庞大规模，当前系统严重依赖云计算（图1(a)-左），因此面临请求负载重、运营成本高以及隐私问题[7]。为缓解这些问题，越来越多的人开始关注设计更小、成本效益更高的LLM，以便利用日益普及的众多多样化边缘计算设备。例如，Meta 提供了多种版本的 Llama [2]，仅有70亿参数，小到可以在边缘设备上运行（图1(a)-中）。尽管这种方法可以提高效率，它通常

与现有在模型和系统层面优化推理的努力形成鲜明对比，我们采取了一条“语义层面”的路径，通过减少大型语言模型的输出以提高吞吐量，*i.e.*, *the LLM creates the response sketch and the SLM fills in the details*。这一观点源于人类回答问题的方式，*i.e.*, 我们只需用几个词就能传达一句话的一般意思。

受到这一想法的启发，我们提出了一种云和边缘上的渐进式推理范式（图 1(b)）。该架构利用边缘上的冗余计算能力，从云端卸载工作负载，同时确保高响应质量。具体来说，在接收到查询后，LLM 首先评估响应的潜在长度。如果响应简短，LLM 将直接输出完整答案。在预计响应较长的情况下，LLM 将生成语义完整但语法最简的草稿，然后边缘上的 SLM 对其进行扩展以提高推理效率。每个短句的语义完整性消除了其扩展的顺序依赖，使 SLM 能够并行处理每个句子，从而打破自回归模型的顺序解码。

Huiyou Zhan, Xuan Zhang, Haisheng Tan, Dongping Yong, Junyang Zhang 和 Xiang-Yang Li 隶属于中国科学技术大学（USTC）LINKE 实验室及中国科学院无线-光通信重点实验室，中国合肥 230027（电子邮件: zh anhuiyou@foxmail.com; xuanzhang@mail.ustc.edu.cn; hstan@ustc.edu.cn; yongdongping@mail.ustc.edu.cn; zhangjunyang@mail.ustc.edu.cn; xiang yangli@ustc.edu.cn）。

韩天隶属于中国合肥中国科学技术大学（USTC），邮编230027（电子邮件: henrytian@ustc.edu.cn）。

渐进推理范式虽然很有前景，但在实际实施中也带来了几个关键挑战：

1) The quality of responses from SLMs may decline; 2) The tradeoff between the brevity and the quality of LLM’s sketches, *i.e.*, while shorter sketches can improve throughput, the semantics expressed may be incomplete; and 3) Additional latency caused by network transmission and edge inference.

In this work, we present PICE, an efficient progressive inference system of LLMs, which synergistically makes use of the resources at the cloud and edge, effectively reducing the workload of LLMs on the cloud, and achieving both high throughput and superior response quality. The effectiveness of PICE stems from four critical designs: a dynamic scheduler taking advantage of progressive inference to optimize throughput, an ensemble learning mechanism aggregating outputs from multiple small models to improve quality, a parallel execution optimization strategy for SLMs to speed up edge inference and a specialized fine-tuning component that enables LLMs to generate concise yet semantically complete sketches. Our main contributions are highlighted as follows.

- We propose and demonstrate the feasibility of synergistic progressive inference at the semantic-level parallel data processing across the cloud and edge. To the best of our knowledge, this is the first attempt to involve semantic optimization with cloud-edge collaboration to enable efficient and reliable LLM serving (Sec. II).
- We design PICE, a novel Progressive InfERENCE engine for LLM serving over Cloud and Edge, achieving high throughput and low latency by employing dynamic scheduling strategies, ensemble learning mechanism and parallel edge inference (Sec. III and Sec. IV).
- By implementing PICE at a testbed with a set of Jetson AGX Orins and a cloud server, we conduct extensive experiments across various LLM benchmarks and popular language models, demonstrating that PICE achieves a throughput enhancement of $1.5\times$ to $2\times$ and a latency reduction by up to 43%, while potentially enhancing the answer quality on several question categories compared to baselines (Sec. V).

II. BACKGROUND AND MOTIVATION

A. LLM Services on the Cloud and Edge

As shown in Table I, the massive size of LLMs, while offering high response quality, entails significant computational resources that can only be met by cloud servers [11]. As more LLM-based services are introduced, the exclusive reliance on the cloud is increasingly unable to meet the requirements for inference efficiency such as the throughput and latency.

Model	Speed(tokens/s)	GPU Memory(GB)	MMLU
Qwen2.5-72B-Instruct	18.19	134.74	86.1
Llama3-70B-Instruct	18.82	130.64	79.5
Qwen2.5-32B-Instruct	22.13	60.11	83.3
Llama3-8B-Instruct	76.5	15.83	66.6
Qwen2.5-7B-Instruct	84.28	14.92	74.2
Qwen2.5-1.5B-Instruct	183.33	3.44	60.9

TABLE I: Performance Comparison. The speed of the models is tested on two NVIDIA A100 GPUs with vLLM framework. The MMLU score is a metric for assessing the performance of language models in multi-task language understanding [12].

Edge computing is a promising solution to address the aforementioned challenges by providing service on edge devices closer to the users [13]. Constrained by computational power and memory, the size of the language models deployed on the edge is typically much smaller than those on the cloud, leading to a decrease in their response quality. For instance, 8B parameters represent the maximum size for Llama to achieve real-time inference on smartphones (*e.g.*, Xiaomi 10) [14], yet its arithmetic reasoning capacity is significantly lower than that of the Llama3-70B model (Table I).

Faced with the tradeoff between inference efficiency and response quality, recent works have introduced a new inference paradigm called routing, which uses two models of different sizes instead of a single model for inference [8], [15]. The smaller model generally has lower inference costs but also lower accuracy than the larger one. The key idea is to identify and route easy queries to the small model so that inference cost can be reduced while maintaining response quality. These approaches typically require training routers from scratch for each scenario, which may lack generalizability. There are also model-level and system-level inference optimization techniques [16], such as model cascading [17], speculative decoding [18], PagedAttention [19], and continuous batching [11], [20], which are mostly applicable to cloud servers and do not fully leverage the computing power of edge devices. The continuous growth of edge intelligent devices motivates us to design a more general cloud-edge LLM serving paradigm.

B. Our idea: Cloud-edge Collaborative Progressive Inference

The effectiveness of Transformer-based LLMs is hindered by the quadratic increase in computational cost relative to sequence length, especially when dealing with long sequences. Due to the autoregressive nature of LLMs, generating one token would require reading the entire KV cache. For the Llama3-8B model with 32k context length, the KV cache can occupy 16GB of space, requiring at least 11 ms for reading, which contributes to more than 50% of the inference latency and limits the overall throughput [21]. This inspires us to investigate the potential for enhancing inference efficiency through reducing the LLM’s response lengths. The feasibility of our idea is supported by the following two observations.

Observation 1: Not all tokens play the same role in expressing semantics. As the redundancy phenomenon in linguistics, the semantic essence of a sentence is often conveyed by just a few words, with others only to refine the grammatical structure. This principle is also evident in the inference of LLMs. For instance, previous work noted that in sentiment analysis tasks, tokens that express strong emotions carry the highest weights in the vector of importance, whereas adpositions such as “and” and “to” receive the least attention [17]. Typically, the performance differences among various models are manifested in these critical tokens. As illustrated in Fig. 2, Qwen2.5-72B exhibits the greater variance in probability distributions for tokens such as “Q”, “exist”, “simultaneously”, and “principle”, compared to its 7B and 1.5B counterparts.

Observation 2: Once key tokens are given, both LLM and SLM exhibit similar conditional probabilities for other

1) SLM 的响应质量可能下降；2) LLM 草图的简短性与质量之间的权衡，*i.e.*，虽然更短的草图可以提高吞吐量，但所表达的语义可能不完整；以及 3) 网络传输和边缘推理造成的额外延迟。

在这项工作中，我们提出了 PICE，一种高效的 LLM 逐步推理系统，该系统协同利用云端和边缘的资源，有效减轻了云端 LLM 的工作负载，并实现了高吞吐量和优越的响应质量。PICE 的有效性来源于四个关键设计：利用逐步推理优化吞吐量的动态调度器、通过集成学习机制聚合多个小模型的输出以提升质量、加速边缘推理的 SLM 并行执行优化策略，以及能够使 LLM 生成简洁但语义完整的草图的专门微调组件。我们的主要贡献总结如下。

- 我们提出并展示了在云端和边缘进行语义层级并行数据处理时协同渐进推理的可行性。据我们所知，这是首次尝试将语义优化与云-边协作结合，以实现高效且可靠的大语言模型（LLM）服务（第 II 节）。
- 我们设计了 PICE，一种新型的用于云端和边缘大规模语言模型（LLM）服务的渐进推理引擎，通过采用动态调度策略、集成学习机制和并行边缘推理，实现了高吞吐量和低延迟（见第 III 节和第 IV 节）。
- 通过在一个由一组 Jetson AGX Orin 和云服务器组成的测试环境中实施 PICE，我们在各种大型语言模型（LLM）基准测试和流行语言模型上进行了广泛实验，结果表明，PICE 可实现 $1.5\times$ 到 $2\times$ 的吞吐量提升，并将延迟减少最多 43%，同时相比基线方法在若干问题类别上可能提高回答质量（第 V 节）。

II. 背景与动机

A. LLM Services on the Cloud and Edge

如表 I 所示，虽然大型语言模型（LLM）提供了高响应质量，但其庞大的规模需要大量计算资源，而这些资源只能由云服务器提供 [11]。随着更多基于 LLM 的服务的推出，仅依赖云计算越来越无法满足诸如吞吐量和延迟等推理效率的要求。

Model	Speed(tokens/s)	GPU Memory(GB)	MMLU
Qwen2.5-72B-Instruct	18.19	134.74	86.1
Llama3-70B-Instruct	18.82	130.64	79.5
Qwen2.5-32B-Instruct	22.13	60.11	83.3
Llama3-8B-Instruct	76.5	15.83	66.6
Qwen2.5-7B-Instruct	84.28	14.92	74.2
Qwen2.5-1.5B-Instruct	183.33	3.44	60.9

表 I：性能比较。模型的速度在使用 vLLM 框架的两块 NVIDIA A100 GPU 上进行测试。MMLU 分数是用于评估语言模型在多任务语言理解中的性能的指标 [12]。

边缘计算是一种有前景的解决方案，通过在更接近用户的边缘设备上提供服务来应对上述挑战 [13]。由于计算能力和内存的限制，部署在边缘的语言模型的规模通常远小于云端模型，这导致其响应质量下降。例如，8B 参数是 Llama 在智能手机上实现实时推理的最大规模（*e.g.*，小米 10）[14]，然而其算术推理能力显著低于 Llama3-70B 模型（表 I）。

面对推理效率和响应质量之间的权衡，最近的研究提出了一种称为路由的新推理范式，该方法在推理时使用两个不同规模的模型，而不是单一模型 [8]，[15]。较小的模型通常推理成本较低，但准确度也低于较大的模型。关键思想是识别并将简单查询路由到小模型，从而在保持响应质量的同时降低推理成本。这些方法通常需要为每种场景从头训练路由器，可能缺乏通用性。还有模型级和系统级的推理优化技术 [16]，例如模型级联 [17]、投机性解码 [18]、PagedAttention [19] 和连续批处理 [11]，[20]，这些大多适用于云服务器，并未充分利用边缘设备的计算能力。边缘智能设备的持续增长促使我们设计更通用的云端-边缘

B. Our idea: Cloud-edge Collaborative Progressive Inference

基于 Transformer 的大型语言模型（LLMs）的有效性受到计算成本相对于序列长度呈二次增长的限制，尤其是在处理长序列时。由于 LLMs 的自回归特性，生成一个 token 需要读取整个 KV 缓存。对于具有 32k 上下文长度的 Llama3-8B 模型，KV 缓存可能占用 16GB 的空间，读取至少需要 11 毫秒，这导致推理延迟超过 50%，并限制了整体吞吐量 [21]。这启发我们通过减少 LLM 的响应长度来提升推理效率。我们的想法的可行性得到以下两个观察结果的支持。

观察 1：并非所有的词元在表达语义时都起到相同的作用。正如语言学中的冗余现象，句子的语义本质通常只通过少数几个词来传达，其余的词仅用于完善语法结构。这一原则在大语言模型（LLM）的推理中也很明显。例如，已有研究指出，在情感分析任务中，表达强烈情感的词元在重要性向量中具有最高权重，而诸如“and”和“to”等介词则受到的关注最少 [17]。通常，各种模型之间的性能差异体现在这些关键词元上。如图 2 所示，Qwen2.5-72B 在“Q”、“exist”、“simultaneously”和“principle”等词元的概率分布中表现出较大的方差，而其 7B 和 1.5B 版本则相对较小。

观察 2：一旦给定关键标记，LLM 和 SLM 对其他的条件概率表现出类似的情况

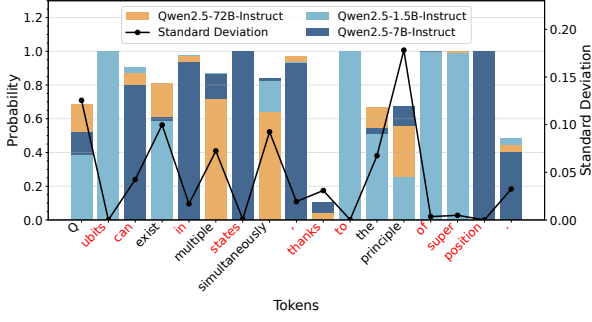


Fig. 2: The conditional probability and variances of the Qwen2.5-72B, Qwen2.5-7B, and Qwen2.5-1.5B models across different tokens. Lower variances indicate closer agreement among the models’ output distributions.

tokens. We analyze the conditional probability distributions of the models when generating the same answer. As illustrated in the Fig. 2, after providing several key tokens (e.g., Q, exist, simultaneously), the variance in the models’ conditional probability distributions for the remaining tokens is relatively low, especially for prepositions like “to” and “of”. This suggests that if the LLM can provide the SLM with key tokens, the SLM’s response quality can be similar to the LLM’s.

Drawing from the above observations, we adopt progressive inference to increase throughput by shortening the responses of LLMs. Specifically, leveraging the response length awareness [22], the LLM addresses long-answer questions through extreme grammatical simplification, as long as the answer is comprehensible regardless of grammatical errors. The simplified answer is subsequently provided to the SLM at edge to be further expanded as a full response. Since each sentence of the sketches is semantically complete, the expansion of different sentences becomes independent. This independence enables SLMs to expand sentences in parallel, breaking autoregressive decoding and thus greatly speeding up the inference process. According to our preliminary experiments (Fig. 3), by employing progressive inference, the generation length of the LLM can be reduced from an average of 500 tokens to 200, thereby increasing the throughput by 1.5-2 $\times$.

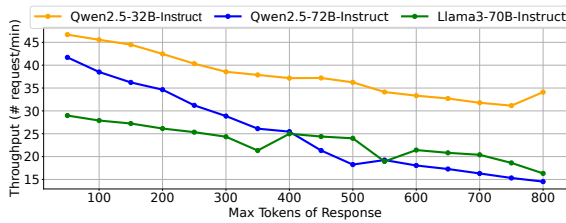


Fig. 3: The horizontal axis represents the max tokens of the LLM’s response, while the vertical axis denotes the throughput of the serving system. The throughput is measured in the number of queries processed per minute (#queries/min).

C. Challenges of Progressive Inference

Challenge 1: Ensuring high response quality. There is a risk of declined response quality due to the limited inference

capability of SLMs, particularly when the sketches provided by the LLM are excessively concise. Ensuring that the SLM can effectively expand and enrich the LLM’s skeletal responses without compromising accuracy or coherence poses a significant challenge.

Challenge 2: Balancing response length and quality. A central challenge involves optimizing the trade-off between the length and quality of sketches. As shown in Fig. 3, shorter sketches by the LLM lead to higher throughput, resulting in fewer tokens for the SLM to utilize as references, potentially reducing response quality. This issue underscores the careful consideration of how much information should be passed from the cloud to the edge.

Challenge 3: Handling latency due to network transmission and edge inference. The integration of network transmission and edge inference adds another layer of complexity. Even though progressive inference only requires transmitting minimal text, network congestion can still adversely affect performance. Additionally, the limited computational capability of edge devices calls for efficient dynamic scheduling of tasks.

Addressing the above challenges requires efforts to enhance the collaborative dynamics between models and optimize the data flow across the cloud and edge, which is crucial for harnessing the full potential of the progressive inference paradigm for a more efficient and responsive inference system.

III. PICE OVERVIEW

Motivated by the huge overheads of serving LLMs and the new optimization opportunities brought by progressive inference, PICE proposes a novel cloud-edge collaborative inference engine to achieve fine-grained throughput improvement efficiently for generic models driven by real-time per-query feedback. Fig. 4 shows a high-level view of PICE and its workflow. The cloud-based LLM inference engine assesses the response length of incoming user requests (❶), providing a full answer directly (❷a), or generating a sketch for complex responses and queuing an expansion task (❷b). For the latter, the scheduler assigns this task to appropriate edge models (❸), and then the system selects the output answer (❹) with the highest confidence to the user (❺). We next introduce the components of PICE in details.

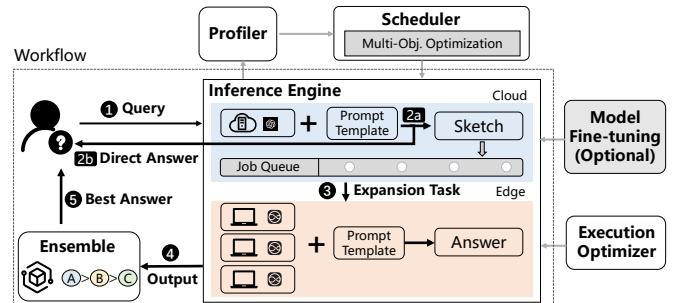


Fig. 4: The overview and workflow of PICE.

Scheduler. The control part of PICE is a scheduler that capitalizes on the advantages of progressive inference to boost system throughput. It dynamically adjusts the sketch length

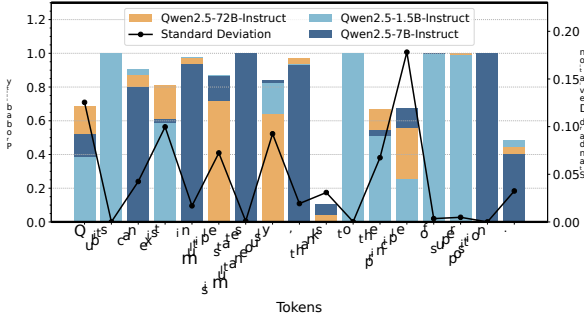


图2: Qwen2.5-72B、Qwen2.5-7B 和 Qwen2.5-1.5B 模型在不同标记上的条件概率和方差。较低的方差表示模型输出分布之间的一致性更高。

标记。我们分析了模型在生成相同答案时的条件概率分布。如图 2 所示，在提供了几个关键标记（e.g.、Q、exist、simultaneously）之后，模型对剩余标记的条件概率分布的方差相对较低，尤其是对于像“to”和“of”这样的介词。这表明，如果大型语言模型（LLM）可以向小型语言模型（SLM）提供关键标记，SLM 的响应质量可以与 LLM 相似。

根据上述观察，我们采用渐进推理来通过缩短大型语言模型（LLM）的回答来提高吞吐量。具体来说，利用对回答长度的感知 [22]，LLM 通过极端语法简化来处理长答案问题，只要答案可理解，不论语法错误与否。随后，将简化后的答案提供给边缘的 SLM 以进一步扩展为完整回答。由于每个草稿句子在语义上都是完整的，不同句子的扩展变得独立。这种独立性使得 SLM 能够并行扩展句子，从而打破自回归解码，并大大加快推理过程。根据我们的初步实验（图 3），通过使用渐进推理，LLM 的生成长度可以从平均 500 个标记减少到 200 个，从而将吞吐量提高 1.5-2 倍。

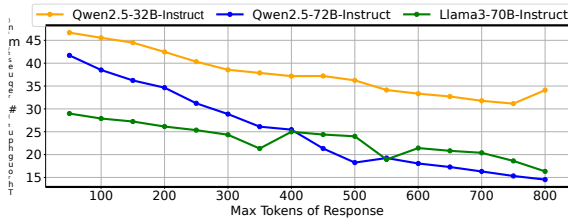


图3: 横轴表示大型语言模型（LLM）响应的最大令牌数，纵轴表示服务系统的吞吐量。吞吐量以每分钟处理的查询数量（#queries/min）来衡量。

C. Challenges of Progressive Inference

挑战 1: 确保高响应质量。由于推理能力有限，存在响应质量下降的风险

SLM 的能力，特别是在 LLM 提供的草图过于简略时。确保 SLM 能够在不影响准确性或连贯性的情况下，有效扩展和丰富 LLM 的框架性回应，是一个重大挑战。

挑战 2: 平衡响应长度和质量。一个核心挑战是优化草图长度与质量之间的权衡。如图 3 所示，LLM 生成的较短草图会提高吞吐量，从而为 SLM 提供的参考令牌较少，这可能会降低响应质量。此问题强调了需要仔细考虑从云端传递到边缘的信息量。

挑战 3: 处理由于网络传输和边缘推理引起的延迟。网络传输与边缘推理的整合增加了另一层复杂性。尽管渐进式推理只需要传输最少的文本，但网络拥堵仍可能对性能产生不利影响。此外，边缘设备有限的计算能力要求对任务进行高效的动态调度。

应对上述挑战需要努力加强模型之间的协作动态，并优化云端与边缘之间的数据流，这对应用渐进推理范式发挥其全部潜力，以实现更高效、更响应的推理系统至关重要。

III. PICE 概述

受到服务大型语言模型（LLM）的高额开销以及渐进式推理带来的新优化机会的驱动，PICE 提出了一个新颖的云-边协作推理引擎，以高效实现针对通用模型的细粒度吞吐量提升，这些模型由基于实时每次查询的反馈驱动。图 4 显示了 PICE 及其工作流程的高层视图。基于云的 LLM 推理引擎评估进入用户请求的响应长度（①），直接提供完整答案（②a），或为复杂响应生成概要并将扩展任务排队（②b）。对于后者，调度器将该任务分配给适当的边缘模型（③），然后系统选择具有最高置信度的输出答案（④）提供给用户（⑤）。接下来我们将详细介绍 PICE 的各个组成部分。

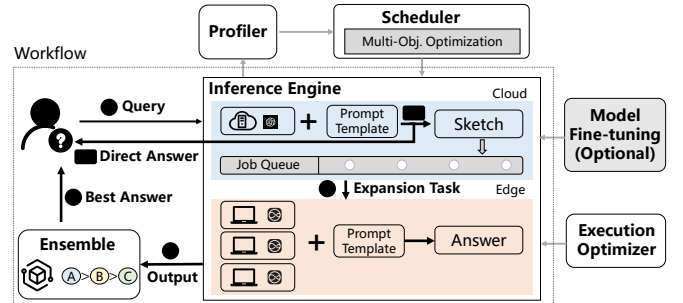


图4: PICE的概览与工作流程。

调度器。PICE 的控制部分是一个调度器，它利用渐进推理的优势来提高系统吞吐量。它会动态调整草图长度

generated by the cloud-based LLM based on complex runtime conditions, intelligently allocates tasks between the cloud and edge to optimize resource utilization, and selects optimal model candidates at the edge for task execution.

Ensemble Learning Mechanism. To offset potential declines in response quality, our system implements an ensemble learning strategy, where multiple smaller models collaborate to generate and select the highest quality response. This method leverages the collective intelligence of diverse models to enhance decision accuracy.

Execution Optimizer. Designed to accelerate inference on edge devices, the execution optimizer is crucial for refining the expansion strategy of sketches. It ensures uniform sequence lengths within batches to maximize processing parallelism, significantly improving overall system efficiency.

Besides the above three key components, there are other components incorporated within our LLM service system.

- **Profiler.** It operates in two phases: offline and runtime. In the offline phase, it conducts device-specific latency estimation. During runtime, it continuously monitors device and server loads, as well as network conditions, providing real-time data to the scheduler for informed decision-making.
- **Inference Engine.** The engine is responsible for executing AI model inferences, divided into cloud and edge parts, and incorporates prompt templates for progressive inference. The cloud component runs LLMs, while the edge component utilizes SLMs across multiple devices for faster computation and improved response time. A job queue buffers tasks from the LLM, which are then optimally distributed by the scheduler to the edge models for processing.
- **Model Fine-tuning (Optional).** To improve the quality of sketches generated by the LLM, we include an optional fine-tuning component. This component uses reinforcement learning from AI feedback to optimize the LLM for creating concise sketches that capture complete semantics with minimal wording. The fine-tuning is performed offline, and the optimized model is then deployed to the cloud for use in cloud-edge progressive inference.

In summary, our system PICE adopts a semantic-level inference acceleration technique that works with existing models and hardware without any changes, which is also compatible with optimization techniques such as speculative decoding [18], PagedAttention [19], and continuous batching [11].

IV. KEY DESIGN

The core of PICE is a novel progressive inference engine, employing efficient and powerful models to serve queries. In this section, we introduce key designs that fully harness the potential of progressive inference.

A. Dynamic Scheduler

1) *Optimization Objective:* We first present the optimization objectives of the dynamic scheduler and formalize the scheduling problem. Let us consider a dataset $D = \{(x_i, y_i)\}_{i=1}^N$ made up of N pairs of inputs x_i and their corresponding outputs y_i . To address the modeling of the

transformation from x to y , we employ a two-stage modeling strategy. Firstly, the input x is transmuted into a more meaningful sketch r by a LLM h , symbolized as $h : x \rightarrow r$. Following this, the subsequent phase consists of employing an SLM s , which takes the extracted representation r and forecasts the output y , expressed as $s : r \rightarrow y$. Through this approach, we essentially break down the complete $x \rightarrow y$ modeling operation as $p(\hat{y}|x) = p_h(r|x) \cdot p_s(y|r)$. Here, $p(\hat{y}|x)$ is the probability model of PICE, serving as an approximation of $p(y|x)$ with error $\delta(\hat{y}, y)$ (δ is the error function).

Therefore, one of our goals is to decrease the error, represented by $E[\delta(\hat{y}, y)]$, through the dual optimization of the SLM s and the sketch r :

$$\sigma^* = \arg \min_{s \in S, r \in R} E[\delta(\hat{y}, y)] = \arg \min_{s \in S, r \in R} E[\delta(s(h(x)), y)] \quad (1)$$

where σ^* denotes the optimal SLM and sketch over all models in set S and all possible sketches R .

Another objective of the system is to maximize throughput. We define the average time $f(l)$ for the cloud-based LLM to process a response of length l . Let c , the cost coefficient, be the ratio between the time for a single execution of the LLM in the cloud and the SLM at the edge. The value of c is not only dependent on the model but also on hardware and software details. Consequently, the average processing time for the SLM to generate a response of length l is $c \cdot f(l)$. The throughput can be formulated as $\max(f(|r|), \Delta(r), \frac{c}{p} \cdot f(l))$, where $\Delta(r)$ is the network latency between edge and cloud, and p denotes the degree of parallelism for edge inference. Additional constraints include maintaining the processing latency for each request below $f(l)$, the latency for cloud inference.

To enable the support of realistic multi-criteria service level agreements (SLAs), we model our scheduling problem as a multi-objective optimization problem. The current set of metrics is defined as

$$M = \{\text{error, throughput, latency, server cost, edge cost}\}$$

We define server cost and edge cost based on the count of generated tokens, respectively. Similar to SPINN [23], we categorize the aforementioned objectives into hard constraints, such as inference latency, and soft constraints, such as maximizing throughput and minimizing errors. To enable the user to specify the importance of each metric, we employ a multi-objective lexicographic formulation [24]:

$$\min_{\sigma} M_i(\sigma), \quad \text{s.t.} \quad M_j(\sigma) \leq M_j(\sigma_j^*) \\ j = 1, 2, \dots, i-1, \quad i > 1, \quad i = 1, 2, \dots, |M|$$

where $M_i \in \{M_1, M_2, \dots, M_{|M|}\}$ is the i -th metric in the ordered tuple of soft targets, i is the position of the metric in the importance sequence, and $M_j(\sigma_j^*)$ represents the optimum of the j -th metric found in the j -th iteration. Under this formulation, we rank the metrics according to their importance as required by the target use case.

2) *Cloud-side scheduling:* We address this optimization problem through multiple steps. When conducting LLM inference in the cloud, we first ensure that the hard constraint of inference latency is met, while enhancing throughput and

由基于云的语言模型根据复杂的运行时条件生成，智能地在云和边缘之间分配任务以优化资源利用，并在边缘为任务执行选择最佳模型候选。

集成学习机制。为了抵消响应质量可能下降的情况，我们的系统实施了集成学习策略，其中多个较小的模型协作生成并选择最高质量的响应。这种方法利用不同模型的集体智能来提高决策准确性。

执行优化器。执行优化器旨在加速边缘设备上的推理，对于优化草图的扩展策略至关重要。它确保批次内序列长度一致，以最大化处理并行性，从而显著提高整体系统效率。

除了上述三个关键组件之外，我们的LLM服务系统中还包含其他组件。

- 分析器。它分两个阶段运行：离线阶段和运行时阶段。在离线阶段，它进行设备特定的延迟估算。在运行时阶段，它持续监控设备和服务器负载，以及网络状况，为调度器提供实时数据，以便做出明智的决策。
- 推理引擎。该引擎负责执行人工智能模型推理，分为云端和边缘部分，并结合提示模板进行渐进式推理。云端组件运行大语言模型（LLM），而边缘组件在多个设备上利用小语言模型（SLM）以实现更快速的计算和更好的响应时间。作业队列缓冲来自大语言模型的任务，然后由调度器将其优化分配给边缘模型进行处理。
- 模型微调（可选）。为了提高大语言模型生成草图的质量，我们包含了一个可选的微调组件。该组件使用来自人工智能反馈的强化学习来优化大语言模型，以生成简明的草图，能够以最少的文字捕捉完整的语义。微调是在离线环境下进行的，然后将优化后的模型部署到云端，用于云边协作的渐进式推理。

总之，我们的系统 PICE 采用了一种语义级的推理加速技术，该技术可以在不更改现有模型和硬件的情况下工作，同时也兼容诸如推测解码 [18]、PagedAttention [19] 和连续批处理 [11] 等优化技术。

四、关键设计

PICE 的核心是一种新型的渐进式推理引擎，采用高效且强大的模型来处理查询。在本节中，我们介绍了充分发挥渐进式推理潜力的关键设计。

A. Dynamic Scheduler

1) *Optimization Objective*: 我们首先提出动态调度器的优化目标，并对调度问题进行形式化。我们考虑一个由 N 对输入 x_i 及其对应输出 y_i 组成的数据集 $D = \{(x_i, y_i)\}_{i=1}^N$ 。为解决对模型的建模问题

x 到 y 的转化，我们采用了两阶段建模策略。首先，输入的 x 通过一个 LLM h 被转化为更有意义的草图 r ，表示为 $h: x \rightarrow r$ 。接下来，下一阶段是使用一个 SLM s ，它以提取的表示 r 为输入并预测输出 y ，表示为 $s: r \rightarrow y$ 。通过这种方法，我们本质上将完整的 $x \rightarrow y$ 建模操作分解为 $p(\hat{y}|x) = p_h(r|x) \cdot p_s(y|r)$ 。其中， $p(\hat{y}|x)$ 是 PICE 的概率模型，作为 $p(y|x)$ 的近似，误差 $\delta(\hat{y}, y)$ (δ 是误差函数)。

因此，我们的目标之一是通过 SLM s 和草图 r 的双重优化来减少由 $E[\delta(\hat{y}, y)]$ 表示的误差：

$$\sigma^* = \arg \min_{s \in S, r \in R} E[\delta(\hat{y}, y)] = \arg \min_{s \in S, r \in R} E[\delta(s(h(x)), y)] \quad (1)$$

其中 σ^* 表示在集合 S 中的所有模型和所有可能的草图 R 上的最优 SLM 和草图。

系统的另一个目标是最大化吞吐量。我们定义云端大型语言模型（LLM）处理长度为 l 的响应的平均时间为 $f(l)$ 。设 c 为成本系数，其定义为在云端执行一次 LLM 与在边缘执行 SLM 所需时间的比值。 c 的值不仅依赖于模型，还取决于硬件和软件的具体情况。因此，边缘 SLM 生成长度为 l 的响应的平均处理时间为 $c \cdot f(l)$ 。吞吐量可表示为 $\max(f(|r|), \Delta(r), \frac{c}{p} \cdot f(l))$ ，其中 $\Delta(r)$ 是边缘与云之间的网络延迟， p 表示边缘推理的并行度。其他约束条件包括保持每个请求的处理延迟低于 $f(l)$ ，即云端推理的延迟。

为了支持现实的多标准服务水平协议（SLA），我们将调度问题建模为多目标优化问题。当前的指标集定义为

$$M = \{\text{错误, 吞吐量, 延迟, 服务器成本, 边缘成本}\}$$

我们分别根据生成的令牌数量定义服务器成本和边缘成本。类似于 SPINN [23]，我们将上述目标分类为硬约束，例如推理延迟，以及软约束，例如最大化吞吐量和最小化错误。为了让用户能够指定每个指标的重要性，我们采用了多目标字典序公式 [24]：

$$\min_{\sigma} M_i(\sigma), \quad s.t. \quad M_j(\sigma) \leq M_j(\sigma_j^*) \\ j = 1, 2, \dots, i-1, \quad i > 1, \quad i = 1, 2, \dots, |M|$$

其中 $M_i \in \{M_1, M_2, \dots, M_{|M|}\}$ 是软目标有序元组中的第 i 个指标， i 是该指标在重要性序列中的位置， $M_j(\sigma_j^*)$ 表示在第 j 次迭代中找到的第 j 个指标的最优值。在此公式下，我们根据目标使用案例的要求对指标进行重要性排序。

2) *Cloud-side scheduling*: 我们通过多个步骤来解决这个优化问题。在云端进行大型语言模型推理时，我们首先确保满足推理延迟的硬性约束，同时提高吞吐量和

response quality. Cloud-side scheduling primarily achieves this by dynamically adjusting the length of the sketch.

Specifically, the total end-to-end latency of a query comprises the latency components from LLM inference, waiting time, network transmission, and SLM inference. We define the LLM inference latency as a function of the output length, f , such that the inference latency for a task r_i on the SLM is given by $\frac{c \cdot f(l_i)}{p}$, where l_i denotes the expected response length for the query. The waiting time represents the total completion time for all tasks in the job queue Q on the edge, calculated as $\frac{\sum_{r_j \in Q} c \cdot f(l_j)}{Np}$, with N representing the number of edge devices. Thus, the constraint for the end-to-end latency can be formulated as follows:

$$f(|r_i|) + \Delta(r_i) + c \cdot f(l_i) + \frac{\sum_{r_j \in Q} c \cdot f(l_j)}{pN} \leq f(l_i) \quad (2)$$

Our scheduling algorithm is inspired by the observation that, similar to humans' ability to estimate the length of an answer based on their understanding of a query, LLMs can also perceive the length of responses in advance, as demonstrated by previous work [22]. In our workflow, the LLM outputs a sketch along with the expected answer length l_i . In our workflow, the LLM outputs a sketch and the expected answer length l_i . Leveraging the measurements of f and c that we established during the offline stage with our profiler, we can estimate the edge inference latency and the waiting time.

We define multiple sketch length levels, from 0 to l_i . Upon receiving a query, we first evaluate the network's conditions and estimate edge inference latency conservatively, setting $p = 1$ by default. If no level above 0 meets the criteria set by inequality (2), we forgo progressive inference and directly request a complete response from the LLM. If the conditions permit, we assess the SLM's ability to select a suitable sketch length, with more capable SLMs potentially opting for shorter lengths to increase throughput. Note that since the sketch length is specified via LLM prompts, r_i may differ by up to 10 tokens from the expected value.

3) *Job dispatching*: In the process for handling queries that require progressive inference, it is crucial to allocate expansion tasks among edge devices in a manner that ensures load balancing. To manage the dynamic nature of these workloads, we implement a multi-list scheduling algorithm that adaptively allocates inference tasks according to the varying demands.

In practical scenarios, response lengths to different queries can vary significantly. This variability, when batching SLM inferences, introduces inefficiencies due to the inclusion of sequences with disparate lengths. Specifically, shorter sequences must wait for longer ones to complete, leading to computational delays. The quadratic time complexity of the inference further exacerbates these inefficiencies, placing a substantial load on the process.

Our job dispatching algorithm, delineated in Algorithm 1, is primarily a multi-list scheduling algorithm that categorizes tasks based on the expected answer length l_i . As queries are received into the job queue Q , they are assigned to specific lists based on their length (Lines 2-6). Once edge devices are

idle, they proactively retrieve a batch of tasks from the longest list to perform inference (Lines 7-11).

Algorithm 1 Job Dispatching

```

1: Initialize lists  $q_1, q_2, \dots, q_n$  for different lengths.
2:  $Q \leftarrow$  List of jobs  $(r_i, l_i)$  waiting for processing
3: while  $Q$  is not empty do
4:   Remove task  $r_i, l_i$  from  $Q$ 
5:   Determine list index  $j$  based on  $l_i$ 
6:   Add  $r_i$  to  $q_j$ 
7: for each edge device  $D$  do
8:   while  $D$  is idle do
9:      $max \leftarrow$  List with the most jobs
10:     $batch \leftarrow$  SelectBatchFrom( $q_{max}$ )
11:     $D.ProcessBatch(batch)$ 
```

4) *Edge-side scheduling*: To cope with the tradeoff between efficiency and response quality, the scheduler must select appropriate SLMs on edge devices for inference. The model selection process includes offline profiling and online selection. Profiling generates a range of differently optimized candidates. When a task arrives, PICE selects a candidate that can meet the targets with the lowest latency.

Offline profiling. The performance of a candidate is jointly decided by the base models and hardware. We initially utilize the profiler to assess the inference latency of candidate SLMs and LLMs when generating sequences of a specified length on edge devices and cloud servers, respectively, and calculated their cost coefficient c . Subsequently, we test the inference latency of the LLM on the cloud server when generating sequences of varying lengths. Both results serve as a basis for estimating the execution time of tasks in the job queue.

Online candidate selection. As shown in Algorithm 2, upon an edge device retrieving a task r_i from the job queue, it first estimates the remaining processing time based on the LLM's anticipated response length l_i and offline profiling data (Line 2). If the total inference time for the task exceeds the threshold $f(l_i)$, thereby violating the hard constraint of the optimizer, a smaller SLM must be selected (Lines 3~4). Conversely, if the total inference time is less than the threshold, a higher quality SLM may be chosen for inference or the current SLM can be retained. To avoid the overhead associated with frequent model switching, the latter process is only conducted when the number of pending tasks in the job queue is less than its maximum (Lines 6~12).

B. Execution Optimizer

Despite the reduced model size on edge devices, the inference latency, limited by their computational and memory capacities, can still present a significant bottleneck for the overall system. Hence, it is crucial to expedite the inference process on edge devices.

We optimize edge inference through semantic-level parallelism strategies. Specifically, the sketch received by the edge inference engine consists of multiple short sentences. As each short sentence is semantically complete, their expansions are

响应质量。云端调度主要通过动态调整草图的长度来实现这一点。

具体而言，一个查询的端到端总延迟包括来自大语言模型（LLM）推理、等待时间、网络传输和小语言模型（SLM）推理的延迟组成部分。我们将 LLM 推理延迟定义为输出长度 f 的函数，使得任务 r_i 在 SLM 上的推理延迟由 $\frac{c \cdot f(l_i)}{p}$ 给出，其中 l_i 表示该查询的预期响应长度。等待时间表示边缘上作业队列 Q 中所有任务的总完成时间，计算公式为 $\frac{\sum_{r_j \in Q} c \cdot f(l_j)}{Np}$ ，其中 N 表示边缘设备的数量。因此，端到端延迟的约束可以表示为如下公式：

$$f(|r_i|) + \Delta(r_i) + c \cdot f(l_i) + \frac{\sum_{r_j \in Q} c \cdot f(l_j)}{pN} \leq f(l_i) \quad (2)$$

我们的调度算法的灵感来自于这样的观察：类似于人类能够根据对查询的理解来估计答案的长度，大型语言模型（LLM）也可以提前感知回答的长度，这一点已被以前的工作[22]所证明。在我们的工作流程中，LLM 会同时输出一个草稿和预期的答案长度 l_i 。在我们的工作流程中，LLM 会输出一个草稿和预期的答案长度 l_i 。利用我们在离线阶段通过分析工具建立的 f 和 c 的测量值，我们可以估计边缘推理延迟和等待时间。

我们定义了多个草图长度等级，从 0 到 l_i 。在收到查询时，我们首先评估网络的条件，并保守地估计边缘推理延迟，默认将 $p =$ 设置为 1。如果没有高于 0 的等级满足不等式 (2) 设定的条件，我们将放弃渐进式推理，直接向 LLM 请求完整响应。如果条件允许，我们会评估 SLM 选择合适草图长度的能力，更有能力的 SLM 可能会选择较短的长度以提高吞吐量。请注意，由于草图长度是通过 LLM 提示指定的， r_i 可能与预期值相差多达 10 个令牌。

3) *Job dispatching*: 在处理需要渐进推理的查询的过程中，至关重要的是以确保负载均衡的方式在边缘设备之间分配扩展任务。为了管理这些工作负载的动态性质，我们实现了一种多列表调度算法，根据不断变化的需求自适应地分配推理任务。

在实际场景中，对不同查询的响应长度可能存在显著差异。这种在批量 SLM 推理时的可变性会引入低效，因为包含了长度不同的序列。具体来说，较短的序列必须等待较长序列完成，从而导致计算延迟。推理的二次时间复杂度进一步加剧了这些低效，使该过程负担沉重。

我们的作业调度算法，如算法1所述，主要是一种多列表调度算法，根据预期答案长度对任务进行分类 l_i 。当查询进入作业队列时 Q ，它们会根据长度分配到特定列表（第2-6行）。一旦边缘设备被

空闲时，它们会主动从最长的列表中取出一批任务来执行推理（第7-11行）。

Algorithm 1 Job Dispatching

```

1: Initialize lists  $q_1, q_2, \dots, q_n$  for different lengths.
2:  $Q \leftarrow$  List of jobs  $(r_i, l_i)$  waiting for processing
3: while  $Q$  is not empty do
4:   Remove task  $r_i, l_i$  from  $Q$ 
5:   Determine list index  $j$  based on  $l_i$ 
6:   Add  $r_i$  to  $q_j$ 
7: for each edge device  $D$  do
8:   while  $D$  is idle do
9:      $max \leftarrow$  List with the most jobs
10:     $batch \leftarrow$  SelectBatchFrom( $q_{max}$ )
11:     $D.ProcessBatch(batch)$ 
```

4) *Edge-side scheduling*: 为了应对效率与响应质量之间的权衡，调度器必须在边缘设备上为推理选择合适的 SLM。模型选择过程包括离线分析和在线选择。分析会生成一系列不同优化的候选模型。当任务到达时，PICE 会选择一個能够以最低延迟满足目标的候选模型。

离线分析。候选模型的性能由基础模型和硬件共同决定。我们最初使用分析器评估候选 SLM 和 LLM 在边缘设备和云服务器上生成指定长度序列时的推理延迟，并计算其成本系数 c 。随后，我们测试 LLM 在云服务器上生成不同长度序列时的推理延迟。两者结果都用于估算作业队列中任务的执行时间。

在线候选选择。如算法2所示，当边缘设备从作业队列中获取任务 r_i 时，它首先根据大模型（LLM）预期的响应长度 l_i 和离线分析数据估算剩余处理时间（第2行）。如果任务的总推理时间超过阈值 $f(l_i)$ ，从而违反优化器的硬约束，则必须选择较小的中型模型（SLM）（第3~4行）。相反，如果总推理时间低于阈值，则可以选择更高质量的 SLM 进行推理，或者保留当前的 SLM。为了避免频繁切换模型带来的开销，后者过程仅在作业队列中待处理任务数量少于其最大值时进行（第6~12行）。

B. Execution Optimizer

尽管边缘设备上的模型尺寸减小，但受限于其计算和内存能力的推理延迟，仍可能对整体系统构成显著瓶颈。因此，加快边缘设备上的推理过程至关重要。

我们通过语义层面的并行策略优化边缘推理。具体来说，边缘推理引擎接收到的草稿由多个简短的句子组成。由于每个简短的句子在语义上是完整的，它们的扩展是

Algorithm 2 Model Selection

```

1: Input:  $r_i, l_i, f(l_i), f(|r_i|)$ 
2: Estimate remaining processing time  $\tau_i$  for current SLM.
3: if  $\tau_i > f(l_i) - f(|r_i|)$  then
4:   Switch to a smaller SLM for  $r_i$ 
5: else
6:   if  $|JobQueue| < \text{maximum}$  then
7:     for each SLM  $m_j$  larger than the current SLM do
8:       Calculate  $\tau'_i$  for  $r_i$ 
9:       if  $\tau'_i < f(l_i) - f(|r_i|)$  then
10:        Upgrade to  $m_j$  for  $r_i$  and break the loop
11:   else
12:     Continue with current SLM for  $r_i$ 

```

independent of each other and can therefore be executed in parallel. The prompt template is as follows.

[System]: I have a question about {query}. The simplification answer is as follows: {sketch}.
 Now, please help me complete and only complete the writing of a short sentence {sentence}. Do not continue with other sentences!
 [Assistant]:

However, higher parallelism is not always preferable. This is due to two factors. (1) Sentence length variability. The differing lengths of sentences can lead to unnecessary waiting when simply batched for processing. (2) Prompt overhead. The need to provide additional sketch information can result in longer prompts for the SLM to handle. Excessive parallelism may result in redundant sketch information in the KV cache, reducing system efficiency.

To tackle the challenges, we adopt a binary-tree-based merging strategy for short sentences. Considering a sketch r_i with k short sentences $(r_{i,1}, r_{i,2}, \dots, r_{i,k})$, we assume that the number of words in a short sentence is positively correlated with the number of words after its expansion. Accordingly, we sort the k short sentences by their word count and combine them into $\lceil \frac{k}{2} \rceil$ groups as follows: $(r_{i,1}, r_{i,k}), (r_{i,2}, r_{i,(k-1)}), \dots, (r_{i, \lceil \frac{k+1}{2} \rceil}, r_{i, \lceil \frac{k+1}{2} \rceil})$. If the current degree of parallelism can still satisfy the hard constraint of inference latency, we recursively apply the aforementioned merging process to increase the throughput of edge devices.

C. Ensemble Learning

The open-source SLMs exhibit diverse strengths and weaknesses due to variations in data, architectures, and hyperparameters, making them complementary to each other. This diversity motivates us to dynamically ensemble these SLMs to generate consistently better responses for each input. A typical ensembling approach involves selecting the optimal candidate from outputs generated by different models, utilizing evaluation metrics such as MLM-Scoring [25] and BERTScore [26] to train a reward model [27]. However, these methods are inapplicable to our system due to the additional overhead caused by training and deploying the reward model. To avoid

the overhead caused by the ensemble, a straightforward way to get the confidence is to use perplexity [28], a metric for assessing a model’s ability to understand and generate human-like language. Perplexity is calculated as the exponential of the average negative log-likelihood of a word sequence.

However, we have observed in practice that perplexity is overly dependent on the model itself. For instance, the Llama3-8B model consistently exhibits higher perplexity across all queries compared to the Qwen2.5-7B, regardless of the actual quality of the responses. To holistically evaluate both the model and the quality of responses, we use a weighted average of text scores and perplexity as the confidence. The text score comprises two components: one is the length of the answer, where more detailed expansions by the SLM receive higher scores; the other is the similarity between the answer and the sketch, with closer matches scoring higher, represented by the rouge-1 score [29]. Thus, the confidence of a sequence can be calculated as follows.

$$\text{con}(\hat{y}) = \alpha_1 \cdot 2^{\frac{1}{N} \sum_{i=1}^N \log_2 p(w_i)} + \alpha_2 \cdot \text{Norm}(|\hat{y}|) + (1 - \alpha_1 - \alpha_2) \cdot \text{Rouge-l}(r, \hat{y}) \quad (3)$$

where N is total number of tokens, w_i is the i -th token in the answer and $p(w_i)$ is probability assigned by the model to the i -th token. The parameters α_1 and α_2 are weights, while $\text{Norm}(|\hat{y}|)$ represents the normalization of the response length. To mitigate the impact of sampling strategies, a single SLM may produce more than one sequence.

D. Model Fine-tuning

The model fine-tuning component in PICE is an optional module designed to enhance the LLM’s ability to generate concise sketches that convey complete semantics with minimal wording. Unlike typical reinforcement learning from human feedback (RLHF), this component introduces a specialized preference labeling algorithm tailored to refine the model’s ability to generate high-quality sketches. As shown in Fig. 5, the fine-tuning process consists of three steps: supervised fine-tuning, reward model training, and reinforcement learning-based fine-tuning.

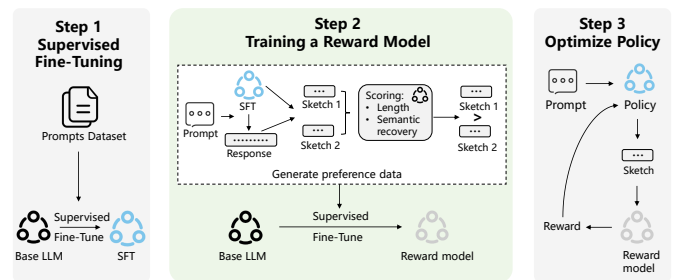


Fig. 5: The design of PICE’s fine-tuning component.

In the first step, the pre-trained LLM is fine-tuned on a labeled dataset to generate concise sketches (e.g., given an input document, generating a summary that captures the main idea of the document), using token-level supervision to produce a supervised fine-tuned (SFT) model π .

Algorithm 2 Model Selection

```

1: Input:  $r_i, l_i, f(l_i), f(|r_i|)$ 
2: Estimate remaining processing time  $\tau_i$  for current SLM.
3: if  $\tau_i > f(l_i) - f(|r_i|)$  then
4:   Switch to a smaller SLM for  $r_i$ 
5: else
6:   if  $|JobQueue| < \text{maximum}$  then
7:     for each SLM  $m_j$  larger than the current SLM do
8:       Calculate  $\tau'_i$  for  $r_i$ 
9:       if  $\tau'_i < f(l_i) - f(|r_i|)$  then
10:        Upgrade to  $m_j$  for  $r_i$  and break the loop
11:   else
12:     Continue with current SLM for  $r_i$ 

```

彼此独立，因此可以并行执行。提示模板如下。

[系统]: 我有一个关于 {查询} 的问题。简化的答案如下: {草图}。现在, 请帮我完成并且只完成一个简短句子的书写 {句子}。不要继续写其他句子! [助手]:

然而，更高的并行性并不总是更可取。这是由于两个因素。(1) 句子长度的可变性。句子长度的差异可能导致在简单批处理时出现不必要的等待。(2) 提示开销。需要额外的草图信息可能导致SLM需要处理更长的提示。过度的并行性可能导致KV缓存中出现冗余的草图信息，从而降低系统效率。

为应对这些挑战，我们采用基于二叉树的短句合并策略。考虑一个包含 k 个短句的草图 r_i ($r_{i,1}, r_{i,2}, \dots, r_{i,k}$)，我们假设短句中的词数与其扩展后的词数呈正相关。因此，我们按词数对 k 个短句进行排序，并将它们合并为 $\lceil \frac{k}{2} \rceil$ 组，如下所示: ($r_{i,1}, r_{i,k}$) , ($r_{i,2}, r_{i,(k-1)}$) , $\dots$, ($r_{i, \lceil \frac{k+1}{2} \rceil}, r_{i, \lceil \frac{k+1}{2} \rceil}$)。如果当前的并行度仍然可以满足推理延迟的硬性约束，我们会递归地应用上述合并过程，以提高边缘设备的吞吐量。

C. Ensemble Learning

开源的SLM由于数据、架构和超参数的差异，表现出多样的优势和劣势，使它们彼此互补。这种多样性促使我们动态集成这些SLM，以便为每个输入生成始终更好的响应。典型的集成方法包括从不同模型生成的输出中选择最优候选，利用如MLM-Scoring [25] 和 BERTScore [26]的评估指标来训练奖励模型 [27]。然而，由于训练和部署奖励模型带来的额外开销，这些方法不适用于我们的系统。为了避免

由集成造成的开销，一个直接获得置信度的方法是使用困惑度 [28]，它是一种评估模型理解和生成类人语言能力的指标。困惑度的计算方法是对一个词序列的平均负对数似然取指数。

然而，我们在实践中观察到，困惑度过度依赖于模型本身。例如，Llama3-8B 模型在所有查询中始终表现出比 Qwen2.5-7B 更高的困惑度，而不管回答的实际质量如何。为了全面评估模型及回答的质量，我们使用文本评分与困惑度的加权平均作为置信度。文本评分由两个部分组成：其一是答案的长度，SLM 对答案的更详细扩展会获得更高的评分；其二是答案与草稿的相似度，相似度越高得分越高，用 rouge-l 分数[29]表示。因此，序列的置信度可以按如下方式计算。

$$\text{con}(\hat{y}) = \alpha_1 \cdot 2^{\frac{1}{N} \sum_{i=1}^N \log_2 p(w_i)} + \alpha_2 \cdot \text{Norm}(|\hat{y}|) + (1 - \alpha_1 - \alpha_2) \cdot \text{Rouge-l}(r, \hat{y}) \quad (3)$$

其中 N 是令牌的总数， w_i 是答案中的第 i 个令牌， $p(w_i)$ 是模型分配给第 i 个令牌的概率。参数 α_1 和 α_2 是权重，而 $\text{Norm}(|\hat{y}|)$ 表示响应长度的归一化。为了减轻采样策略的影响，单个 SLM 可能生成多个序列。

D. Model Fine-tuning

PICE中的模型微调组件是一个可选模块，旨在增强大语言模型（LLM）生成简明草图的能力，使其以最少的文字传达完整的语义。与典型的基于人类反馈的强化学习（RLHF）不同，该组件引入了一种专门的偏好标注算法，专门用于优化模型生成高质量草图的能力。如图5所示，微调过程包括三个步骤：监督微调、奖励模型训练和基于强化学习的微调。

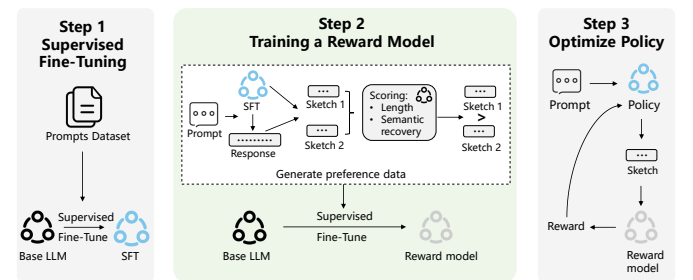


图5: PICE微调组件的设计。

在第一步中，预训练的大型语言模型（LLM）在标注数据集上进行微调，以生成简明的草图（例如，给定一个输入文档，生成一个抓住文档主要思想的摘要），使用逐令牌监督来生成一个监督微调（SFT）模型 π_0 。

In the second step, our goal is to generate a reward model that takes in a sequence of sketches, and returns a scalar reward which should numerically represent our preference for a good sketch. The training dataset of prompt-generation pairs for the reward model is generated by the SFT model and a preference labeling algorithm. Given an input x , PICE first obtains a full answer y from the SFT model π . Then, the query x and answer y are provided to the model π , which generates a pair of sketches $(r1, r2)$ for y . The input and sketches are sent to the preference labeling algorithm to rate which sketch is better according to the following criteria.

- The shorter the length of the sketch (l_r), the higher the score it receives.
- The base LLM expands the sketch r into a full answer $\hat{y}$, which is compared to the answer y from the SFT model. Higher similarity indicates a higher score for the sketch.

The score of a sketch can be calculated as follows: $\beta_1 \cdot \frac{1}{l_r} + \beta_2 \cdot \text{Rouge-l}(\hat{y}, y)$. The parameters β_1 and β_2 are weights. These elements form a triplet dataset $D = \{(x, r_w, r_l)\}$, where r_w and r_l are the preferred (i.e., higher-scoring) and non-preferred sketches, respectively. The reward model (RM) R_ϕ is trained by minimizing the following loss:

$$\mathcal{L}_R(\phi) = -\mathbb{E}_{(x, r_w, r_l) \sim D} [\log \sigma(R_\phi(x, r_w) - R_\phi(x, r_l))],$$

where σ is the sigmoid function.

In the third step, a policy π_θ^{RL} is initialized with the weights of the SFT model and then refined using reinforcement learning to maximize the reward from the RM, which serves as a proxy for sketch preferences. To prevent excessive divergence from the original SFT policy π , a Kullback-Leibler (KL) divergence [30] term D_{KL} is incorporated into the objective. This term, controlled by the hyperparameter γ [31], ensures that π_θ^{RL} does not produce outputs that, although highly rewarded by the RM, have low quality or unnatural language [32]. The optimization objective is defined as follows:

$$J(\theta) = \mathbb{E}_{r \sim \pi_\theta(\cdot|x)} [(1 - \gamma)R_\phi(r|x) - \gamma D_{KL}(\pi_\theta^{RL}(r|x) \parallel \pi(r|x))].$$

V. EVALUATION

We evaluate PICE through extensive testbed-based experiments with various LLM benchmarks and popular language models. We first assess its overall performance in enhancing inference efficiency and ensuring response quality (Sec. V-A). Next, we examine its three core components within the cloud-edge progressive inference framework ((Sec. V-C)). Additionally, we analyze the impact of hyperparameters on system performance (Sec. V-D). The main takeaways are:

- PICE achieves $1.5\times \sim 2\times$ throughput improvement and up to 43% latency reduction over baseline methods.
- PICE effectively maintains response quality, and even enhances it on several question categories.
- The strategic optimizations within the LLM serving system enable PICE to effectively balance inference efficiency and quality in various scenarios.

A. Methodology

Testbed setup. Our experimental infrastructure comprises eight devices, including four Jetson AGX Orin units and a cloud server equipped with four NVIDIA A100 Tensor Core GPUs and 112 CPU cores, which serve as the primary inference devices. The detailed specifications of these devices are provided in Table. II. For the software environment, the cloud inference engine operates on the vLLM framework [19], while the edge inference engine is implemented by PyTorch [33] and Transformers [34].

	Device	Memory	Memory bandwidth	AI performance
Edge	Jetson AGX Orin	64GB	204.8GB/s	137.5 TFLOPS
Cloud	NVIDIA A100	80GB	1935GB/s	624 TFLOPS

TABLE II: Physical devices.

Metrics and benchmarks. The performance of PICE is evaluated using a series of Llama and Qwen models, as detailed in Table I. The key metric for inference efficiency is throughput, defined as the number of requests processed by the system per minute. Additionally, we measure average end-to-end latency, which encompasses inference latencies at both cloud and edge levels, waiting time, and data transfer latency. To evaluate response quality, we adopt two LLM-based evaluation frameworks: FastChat [35] and LLMZoo [36]. These frameworks involve presenting a series of open-ended questions to an LLM judge (e.g., GPT-3.5 turbo) to score and rank responses based on quality. We specifically utilize two datasets, MT-bench and Vicuna-bench [35], designed to assess the instruction-following abilities of chatbots and differentiate them based on their core capabilities (e.g., reasoning and mathematics).

Baselines. We compare the performance in terms of inference efficiency and answer quality of PICE with the baselines.

- Cloud-only. All queries are serviced by language models deployed on the cloud with the vLLM framework [19].
- Edge-only. In this case, all queries are handled by SLMs ($\leq 8B$) that are locally deployed on edge devices. Each edge device hosts a model, and queries are dispatching in a load-balanced manner.
- Routing [8]. It uses a router to dynamically allocate queries to either a small at edge or large model at cloud based on the predicted query difficulty.

B. Overall Performance

Throughput and latency. To assess throughput, we configure the cloud server's maximum batch size for the Qwen2.5-72B-Instruct model at 20, with other devices and models proportionally adjusted. The requests per minute (RPM) is 1.5 times the maximum batch size. The throughput and end-to-end latency of LLM inference are shown in Table. III. We have the following observations. First, PICE is potential and beneficial for large language model deployment. PICE achieves significantly higher inference throughput and lower inference latency than baseline methods by leveraging edge computing resources. For the Llama3-70B model, PICE delivers an average throughput of 25.98 requests per

在第二步中，我们的目标是生成一个奖励模型，该模型接收一系列草图，并返回一个标量奖励，该奖励应数值上表示我们对优秀草图的偏好。奖励模型的提示-生成对训练数据集由SFT模型和偏好标注算法生成。给定输入 x ，PICE首先从SFT模型 π 获取完整答案 y 。然后，将查询 x 和答案 y 提供给模型 π ，该模型生成一对草图 $(r1, r2)$ 用于 y 。输入和草图会被发送到偏好标注算法，以根据以下标准评定哪幅草图更好。

- 草图 (l_r) 的长度越短，得分就越高。
- 基础大语言模型将草图 r 扩展为完整答案 $\hat{y}$ ，然后与 SFT 模型的答案 y 进行比较。相似度越高，草图的得分越高。

草图的得分可以按如下方式计算： $\beta_1 \cdot \frac{1}{l_r} + \beta_2 \cdot \text{Rouge-l}(\hat{y}, y)$ 。参数 β_1 和 β_2 是权重。这些元素形成一个三元组数据集 $D = \{(x, r_w, r_l)\}$ ，其中 r_w 和 r_l 分别是优选（即得分较高）和非优选的草图。奖励模型（RM） R_ϕ 通过最小化以下损失进行训练：

$$\mathcal{L}_R(\phi) = -\mathbb{E}_{(x, r_w, r_l) \sim D} [\log \sigma(R_\phi(x, r_w) - R_\phi(x, r_l))],$$

其中 σ 是 sigmoid 函数。

在第三步中，策略 π_θ^{RL} 使用 SFT 模型的权重进行初始化，然后通过强化学习进行优化，以最大化来自 RM 的奖励，RM 充当草图偏好的代理。为了防止与原始 SFT 策略 π 产生过度偏离，目标函数中引入了 Kullback-Leibler (KL) 散度 [30] 项 D_{KL} 。该项由超参数 γ [31] 控制，以确保 π_θ^{RL} 不会产生尽管 RM 给了高奖励但质量低或语言不自然的输出 [32]。优化目标定义如下：

$$J(\theta) = \mathbb{E}_{r \sim \pi_\theta(\cdot|x)} [(1 - \gamma)R_\phi(r|x) - \gamma D_{KL}(\pi_\theta^{RL}(r|x) \parallel \pi(r|x))].$$

五、评估

我们通过基于测试平台的大量实验，使用各种大型语言模型基准和流行语言模型来评估 PICE。我们首先评估其在提升推理效率和保证响应质量方面的整体表现（第 V-A 节）。接着，我们在云-边缘渐进推理框架中检查其三个核心组件（第 V-C 节）。此外，我们分析了超参数对系统性能的影响（第 V-D 节）。主要结论如下：

- PICE 相比基线方法实现了 $1.5 \times \sim 2 \times$ 的吞吐量提升，并将延迟最多降低 43%。
- PICE 有效地保持了响应质量，甚至在若干问题类别上提升了响应质量。
- LLM 服务系统内的战略优化使 PICE 能够在各种场景下有效平衡推理效率和质量。

A. Methodology

测试平台设置。我们的实验基础设施包括八台设备，其中包含四台 Jetson AGX Orin 单元和一台配备四块 NVIDIA A100 Tensor Core GPU 及 112 个 CPU 核心的云服务器，这些设备作为主要的推理设备。表 II 提供了这些设备的详细规格。在软件环境方面，云端推理引擎运行在 vLLM 框架 [19] 上，而边缘推理引擎由 PyTorch [33] 和 Transformers [34] 实现。

	Device	Memory	Memory bandwidth	AI performance
Edge	Jetson AGX Orin	64GB	204.8GB/s	137.5 TFLOPS
Cloud	NVIDIA A100	80GB	1935GB/s	624 TFLOPS

表 II：物理设备。

指标与基准。通过一系列 Llama 和 Qwen 模型评估 PICE 的性能，如表 I 所示。推理效率的关键指标是吞吐量，定义为系统每分钟处理的请求数量。此外，我们还测量平均端到端延迟，包括云端和边缘的推理延迟、等待时间以及数据传输延迟。为了评估响应质量，我们采用两个基于 LLM 的评估框架：FastChat [35] 和 LLMZoo [36]。这些框架涉及向 LLM 评审（例如 GPT-3.5 turbo）提供一系列开放式问题，并根据质量对响应进行评分和排名。我们特别使用两个数据集，MT-bench 和 Vicuna-bench [35]，用于评估聊天机器人的指令执行能力，并根据其核心能力（e.g.、推理和数学）进行区分。

基线。我们将 PICE 的推理效率和答案质量与基线进行比较。

- 仅限云端。所有查询均由部署在云端的基于 vLLM 框架 [19] 的语言模型处理。
- 仅限边缘。在这种情况下，所有查询都由部署在边缘设备上的 SLM ($\leq 8B$) 本地处理。每个边缘设备都承载一个模型，并且查询以负载均衡的方式分发。
- 路由 [8]。它使用路由器根据预测的查询难度，将查询动态分配到边缘的小模型或云端的大模型。

B. Overall Performance

吞吐量和延迟。为了评估吞吐量，我们将云服务器上 Qwen2.5-72B-Instruct 模型的最大批量大小配置为 20，其他设备和模型按比例进行调整。每分钟请求数 (RPM) 为最大批量大小的 1.5 倍。LLM 推理的吞吐量和端到端延迟如表 II I 所示。我们有以下观察结果。首先，PICE 对大型语言模型部署具有潜力和益处。PICE 利用边缘计算资源实现了显著更高的推理吞吐量和更低的推理延迟，相较于基线方法。对于 Llama3-70B 模型，PICE 提供的平均吞吐量为每

Methods	Qwen2.5-72B-Instruct		Llama3-70B-Instruct		Qwen2.5-32B-Instruct		Llama3-8B-Instruct		Qwen2.5-7B-Instruct		Qwen2.5-1.5B-Instruct	
	Throughput Latency		Throughput Latency		Throughput Latency		Throughput Latency		Throughput Latency		Throughput Latency	
Cloud-only	14.891	138.62	16.33	121.54	32.13	72.32	75.51	28.57	88.33	30.88	148.12	23.71
Edge-only	OOM	OOM	OOM	OOM	OOM	OOM	6.03	804.21	6.68	801.23	21.20	210.38
Routing	14.86	145.04	13.79	143.94	30.04	88.57	69.55	74.75	69.55	68.66	133.31	41.28
PICE	21.24	97.34	25.98	75.15	34.81	61.22	70.48	30.21	84.98	31.78	140.86	26.19

TABLE III: Inference efficiency comparison. In the table, the models refer to those running in the cloud. For routing and PICE, the SLM at edge is any model with fewer parameters than the cloud model. Throughput is measured in the number of queries processed per minute (#queries/min), and latency is measured in seconds (s). “OOM” stands for “Out of Memory”.

Methods	Metrics	Overall	Question categories									
			generic	knowledge	roleplay	fermi	coding	math	writing	reasoning	stem	humanities
Cloud-only	Overall score	8.03	7.90	8.20	8.12	8.20	8.00	8.50	7.81	8.33	8.20	7.83
	Diversity rank	1.82	1.90	1.80	1.59	1.50	1.50	1.36	1.56	1.67	2.50	2.78
	Relevance rank	1.67	1.40	1.00	1.06	1.10	1.00	1.00	1.17	1.00	4.00	4.00
	Immersion rank	1.60	1.60	1.00	1.35	1.10	1.25	1.09	1.17	1.00	2.90	3.56
	Coherence rank	1.55	1.30	1.00	1.18	1.00	1.25	1.00	1.17	1.17	3.60	2.89
	Integrity rank	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Edge-only	Overall score	7.38	8.00	7.60	7.00	7.60	7.27	7.70	7.75	6.83	7.60	8.00
	Diversity rank	1.81	1.70	1.60	1.88	1.40	1.81	2.18	2.28	2.67	1.40	1.22
	Relevance rank	2.01	1.20	1.00	2.24	1.60	2.63	3.36	2.50	3.50	1.10	1.00
	Immersion rank	1.99	1.20	1.20	2.41	1.40	2.44	2.82	2.33	3.83	1.20	1.11
	Coherence rank	1.90	1.60	1.00	1.88	1.40	2.25	2.64	2.22	3.83	1.00	1.22
	Integrity rank	1.44	1.00	1.00	1.38	1.00	1.56	2.36	1.83	2.33	1.00	1.00
Routing	Overall score	7.52	7.70	7.70	7.82	7.90	7.67	8.30	8.00	6.67	7.80	7.33
	Diversity rank	1.81	2.00	2.10	1.76	1.50	1.94	2.09	1.67	2.33	1.40	1.56
	Relevance rank	1.71	1.20	1.30	1.47	1.70	2.00	2.36	1.67	2.33	1.20	1.44
	Immersion rank	1.90	1.60	1.90	1.47	2.20	1.81	3.00	1.56	2.83	1.50	1.22
	Coherence rank	1.75	2.40	1.00	1.24	1.90	2.06	2.36	1.28	2.83	1.20	1.44
	Integrity rank	1.32	1.00	1.00	1.00	1.30	1.31	1.27	1.00	2.00	1.60	1.33
PICE	Overall score	8.37	8.30	8.60	8.65	8.60	7.00	8.10	8.12	9.00	8.80	8.50
	Diversity rank	1.70	1.40	1.70	2.00	2.10	1.44	1.18	1.39	2.00	2.20	2.44
	Relevance rank	1.10	1.00	1.00	1.12	1.10	1.00	1.00	1.11	1.67	1.50	1.56
	Immersion rank	1.21	1.20	1.00	1.59	1.10	1.13	1.09	1.11	1.67	1.20	1.00
	Coherence rank	1.13	1.20	1.00	1.18	1.30	1.13	1.00	1.11	1.67	1.40	1.00
	Integrity rank	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

TABLE IV: Response quality comparison. The overall scores are provided by FastChat [35], with values of (1-10), where higher scores indicate higher quality. In addition to a general metric, LLMZoo provides five detailed metrics [36]. For the same question, LLMZoo ranks the responses provided by the four methods across these metrics, with values of (1-4), where a rank of 1 indicates the best performance. Both evaluation frameworks use GPT-3.5 Turbo as the LLM judge.

minute, approximately 1.59 times that of Cloud-only and 2.32 times that of the Routing method, while also reducing latency by about 38% to 58%. Similar improvements are noted for the Qwen2.5-72B model. Secondly, for the Qwen2.5-32B model, the performance of PICE is on par with Cloud-only. This is attributed to the model’s poor ability to predict response lengths, often underestimating them, leading PICE to favor not engaging the progressive inference mode. Thirdly, for smaller models such as the Llama3-8B, the efficiency of PICE is slightly inferior to Cloud-only but superior to both Routing and Edge-only. This is because the size difference between small edge models and cloud models is insufficient, making edge inference a bottleneck for the system. Therefore, we recommend that the size of the LLM in PICE should be more than ten times that of the SLM.

Response quality. As shown in Table IV, PICE achieves higher overall scores than the baselines, indicating that PICE, due to its integration of multiple models’ outputs, improves response quality compared to using an LLM alone, particularly

for knowledge, roleplay, and reasoning questions. However, PICE performs less well on math and coding tasks than the Cloud-only method, due to the difficulty in fully capturing essential meanings in the preliminary sketches generated by the LLM. Additionally, PICE outperforms the baseline on coherence, diversity, immersion, integrity, and relevance metrics, achieving the best average ranking, especially in terms of integrity across all tasks. This performance suggests that the two-stage approach of progressive inference, while reliant on the quality of LLM-generated sketches, effectively maintains completeness and promotes diversity in the outputs.

C. Performance Breakdown

We evaluate the benefits brought by PICE’s key designs separately. The experiments are performed with LLama3-70B-Instruct in the cloud, and the three SLMs in the edge.

Efficiency improvement with dynamic scheduler. The dynamic scheduler in PICE balances inference efficiency and response quality by adapting to runtime conditions. In contrast,

Methods	Qwen2.5-72B-Instruct		Llama3-70B-Instruct		Qwen2.5-32B-Instruct		Llama3-8B-Instruct		Qwen2.5-7B-Instruct		Qwen2.5-1.5B-Instruct	
	Throughput Latency		Throughput Latency		Throughput Latency		Throughput Latency		Throughput Latency		Throughput Latency	
Cloud-only	14.891	138.62	16.33	121.54	32.13	72.32	75.51	28.57	88.33	30.88	148.12	23.71
Edge-only	OOM	OOM	OOM	OOM	OOM	OOM	6.03	804.21	6.68	801.23	21.20	210.38
Routing	14.86	145.04	13.79	143.94	30.04	88.57	69.55	74.75	69.55	68.66	133.31	41.28
PICE	21.24	97.34	25.98	75.15	34.81	61.22	70.48	30.21	84.98	31.78	140.86	26.19

表 III: 推理效率比较。表中所指的模型为在云端运行的模型。对于路由和 PICE, 边缘的 SLM 是指参数少于云端模型的任意模型。吞吐量以每分钟处理的查询数量 (#queries/min) 来衡量, 延迟以秒 (s) 为单位。“OOM”表示“内存不足”。

Methods	Metrics	Overall	Question categories									
			generic	knowledge	roleplay	fermi	coding	math	writing	reasoning	stem	humanities
Cloud-only	Overall score	8.03	7.90	8.20	8.12	8.20	8.00	8.50	7.81	8.33	8.20	7.83
	Diversity rank	1.82	1.90	1.80	1.59	1.50	1.50	1.36	1.56	1.67	2.50	2.78
	Relevance rank	1.67	1.40	1.00	1.06	1.10	1.00	1.00	1.17	1.00	4.00	4.00
	Immersion rank	1.60	1.60	1.00	1.35	1.10	1.25	1.09	1.17	1.00	2.90	3.56
	Coherence rank	1.55	1.30	1.00	1.18	1.00	1.25	1.00	1.17	1.17	3.60	2.89
	Integrity rank	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
Edge-only	Overall score	7.38	8.00	7.60	7.00	7.60	7.27	7.70	7.75	6.83	7.60	8.00
	Diversity rank	1.81	1.70	1.60	1.88	1.40	1.81	2.18	2.28	2.67	1.40	1.22
	Relevance rank	2.01	1.20	1.00	2.24	1.60	2.63	3.36	2.50	3.50	1.10	1.00
	Immersion rank	1.99	1.20	1.20	2.41	1.40	2.44	2.82	2.33	3.83	1.20	1.11
	Coherence rank	1.90	1.60	1.00	1.88	1.40	2.25	2.64	2.22	3.83	1.00	1.22
	Integrity rank	1.44	1.00	1.00	1.38	1.00	1.56	2.36	1.83	2.33	1.00	1.00
Routing	Overall score	7.52	7.70	7.70	7.82	7.90	7.67	8.30	8.00	6.67	7.80	7.33
	Diversity rank	1.81	2.00	2.10	1.76	1.50	1.94	2.09	1.67	2.33	1.40	1.56
	Relevance rank	1.71	1.20	1.30	1.47	1.70	2.00	2.36	1.67	2.33	1.20	1.44
	Immersion rank	1.90	1.60	1.90	1.47	2.20	1.81	3.00	1.56	2.83	1.50	1.22
	Coherence rank	1.75	2.40	1.00	1.24	1.90	2.06	2.36	1.28	2.83	1.20	1.44
	Integrity rank	1.32	1.00	1.00	1.00	1.30	1.31	1.27	1.00	2.00	1.60	1.33
PICE	Overall score	8.37	8.30	8.60	8.65	8.60	7.00	8.10	8.12	9.00	8.80	8.50
	Diversity rank	1.70	1.40	1.70	2.00	2.10	1.44	1.18	1.39	2.00	2.20	2.44
	Relevance rank	1.10	1.00	1.00	1.12	1.10	1.00	1.00	1.11	1.67	1.50	1.56
	Immersion rank	1.21	1.20	1.00	1.59	1.10	1.13	1.09	1.11	1.67	1.20	1.00
	Coherence rank	1.13	1.20	1.00	1.18	1.30	1.13	1.00	1.11	1.67	1.40	1.00
	Integrity rank	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00

表 IV: 响应质量比较。总评分由 FastChat [35] 提供, 评分范围为 (1-10), 分值越高表示质量越高。除了整体指标外, L LMZoo 提供了五项详细指标 [36]。对于相同的问题, LLMZoo 对四种方法提供的回答在这些指标上进行排名, 排名范围为 (1-4), 其中排名 1 表示表现最佳。两个评估框架均使用 GPT-3.5 Turbo 作为 LLM 评判。

每分钟, 大约是仅云方案的1.59倍, 是路由方法的2.32倍, 同时延迟也降低了约38%到58%。对于Qwen2.5-72B模型也有类似的改进。其次, 对于Qwen2.5-32B模型, PICE的性能与仅云方案相当。这归因于该模型预测响应长度的能力较差, 经常低估响应长度, 导致PICE倾向于不启用渐进推理模式。第三, 对于像Llama3-8B这样的小模型, PICE的效率略低于仅云方案, 但优于路由和仅边缘方案。这是因为小型边缘模型与云模型之间的规模差异不足, 使边缘推理成为系统的瓶颈。因此, 我们建议PICE中LLM的规模应超过SLM的十倍以上。

响应质量。如表 IV 所示, PICE 的整体得分高于基线方法, 这表明由于整合了多个模型的输出, PICE 提高了响应质量, 相比单独使用大型语言模型尤其如此

对于知识、角色扮演和推理问题, PICE表现不错。然而, 由于难以在LLM生成的初步草图中完全捕捉核心含义, PICE在数学和编码任务上的表现不如仅云方法。此外, PICE在连贯性、多样性、沉浸感、完整性和相关性指标上均优于基线方法, 取得了最佳平均排名, 尤其是在所有任务中的完整性方面。这一表现表明, 逐步推理的两阶段方法虽然依赖于LLM生成草图的质量, 但能够有效保持完整性并促进输出的多样性。

C. Performance Breakdown

我们分别评估了 PICE 关键设计所带来的收益。实验在云端使用 LLama3-70B-Instruct 进行, 在边缘使用三个 SLM 进行。通过动态调度器提高效率。PICE 中的动态调度器通过适应运行时条件, 在推理效率和响应质量之间实现平衡。相比之下,

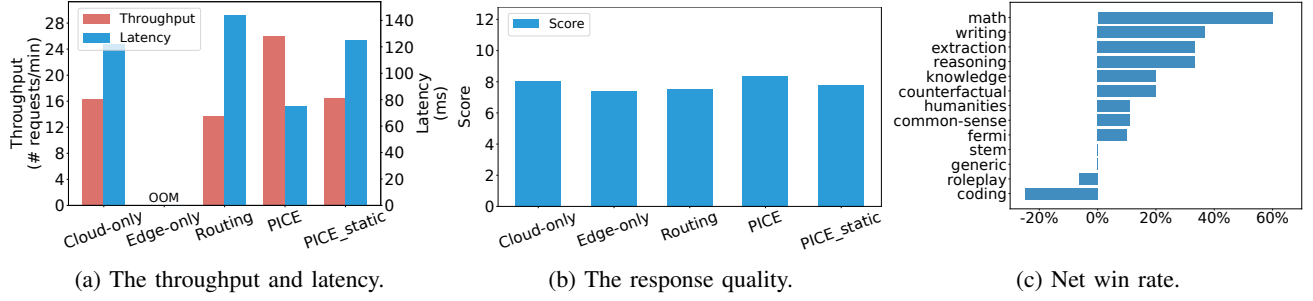


Fig. 6: The efficiency and response quality improvement with dynamic scheduler. Net win rate in (c) is the difference between the fraction of questions that PICE has better and worse answers than PICE with static scheduler.

static scheduling adheres to predefined rules and parameters, allocating requests between cloud and edge devices based solely on predicted answer lengths. As depicted in Fig. 6(a), while PICE with static scheduling modestly improves throughput compared to the Cloud-only method, dynamic scheduling significantly enhances throughput by an additional 50%. Additionally, dynamic scheduling significantly reduces latency compared to static scheduling and other methods like Cloud-only or Routing, demonstrating its effectiveness in balancing throughput with response time.

As shown in Fig. 6(b), response quality is improved by 6.0%, even surpassing that of standalone LLMs, as PICE produces more detailed and comprehensive answers. As detailed in Fig. 6(c), the dynamic scheduler markedly enhances answer quality in 69% of question categories, with significant improvements in mathematics and writing tasks, where it achieves net win rates of 60% and 36.8%, respectively.

Latency reduction with execution optimizer. The execution optimizer capitalizes on semantic-level parallelism to accelerate edge inference. In Fig. 7(a), we analyze the optimal parallelism of different tasks based on the sketch length. For tasks such as generic and roleplay, parallelism increases with sketch length, peaks around 500 tokens, and then declines, indicating benefits from longer sketches until limited by edge device memory. Conversely, tasks like common-sense and math maintain consistently low parallelism, as their inherently shorter responses do not require high parallelism.

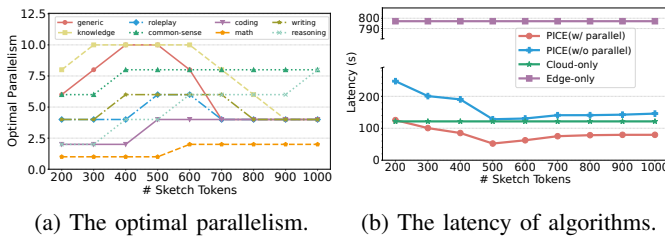


Fig. 7: The impact of parallel generating.

As depicted in Fig. 7(b), the parallel mechanism decreases the inference latency of the PICE system at least 62s. When the sketch token count is below 500, increasing the length of sketches enhances the parallelism in edge inference, significantly reducing latency. However, once the count exceeds 500 tokens, the latency reduction with PICE becomes less marked. This occurs because the required KV cache for the

prompts enlarges, and the limited memory of edge devices cannot support further increases in parallelism. These results emphasize the need to adjust parallelism based on task type and sketches to optimize inference efficiency.

Quality improvement with ensemble learning. As shown in Fig. 8, the confidence rankings of SLMs differ significantly across various question categories, underscoring each model’s specialized capabilities. Leveraging this diversity, PICE adopts an ensemble learning approach to select the best answer. As evidenced by Fig. 9, this method enhances response quality in nearly all categories, except for coding and counterfactual scenarios, with the most substantial improvements noted in roleplay (13.95%) and knowledge (11.69%).

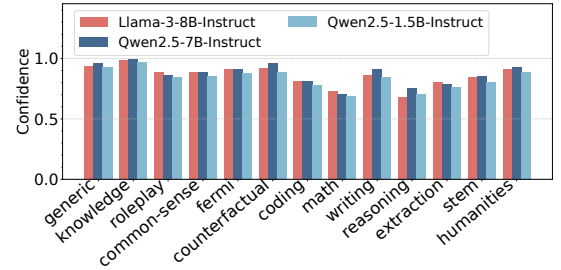


Fig. 8: The confidence of SLMs.

Furthermore, the overall response quality is improved by 2.8%, demonstrating the effectiveness of ensemble learning in combining the strengths of individual models. The notable gains in categories like roleplay and knowledge highlight the ensemble’s ability to better capture nuanced and detailed information, ensuring higher-quality outputs in tasks that benefit from diverse model capabilities.

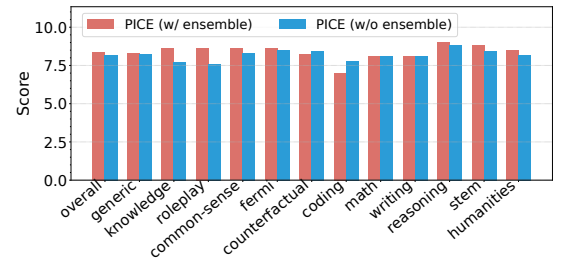


Fig. 9: Impact of ensemble learning on response quality.

Performance improvement with model Fine-tuning. The

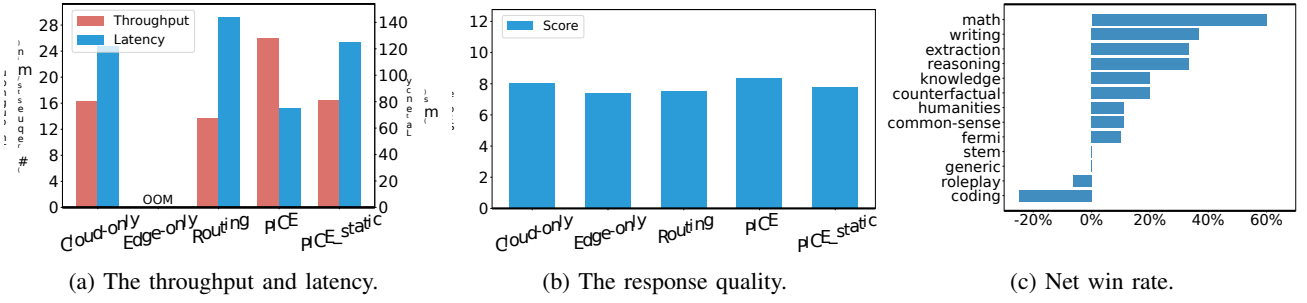


图6: 使用动态调度器时效率和响应质量的提升。(c) 中的净胜率是PICE相对于使用静态调度器时答案更好和更差的问题比例的差值。

静态调度遵循预定义的规则和参数, 仅根据预测的答案长度在云端和边缘设备之间分配请求。如图6(a)所示, 与仅云端方法相比, 采用静态调度的PICE稍微提升了吞吐量, 而动态调度则显著将吞吐量提高了额外50%。此外, 与静态调度及其他方法(如仅云端或路由)相比, 动态调度显著降低了延迟, 证明其在平衡吞吐量和响应时间方面的有效性。

如图6(b)所示, 响应质量提高了6.0%, 甚至超过了独立大语言模型(LLM)的表现, 因为PICE能够生成更详细和更全面的答案。如图6(c)所示, 动态调度器显著提高了69%问题类别的答案质量, 在数学和写作任务中表现尤为突出, 净胜率分别达到60%和36.8%。

使用执行优化器降低延迟。执行优化器利用语义级并行性来加速边缘推理。在图7(a)中, 我们根据草图长度分析了不同任务的最佳并行度。对于通用和角色扮演等任务, 并行度随草图长度增加, 在大约500个标记时达到峰值, 然后下降, 这表明随着草图变长可以获得好处, 直到受到边缘设备内存的限制。相反, 常识和数学等任务保持始终较低的并行度, 因为它们本质上较短的响应不需要高并行度。

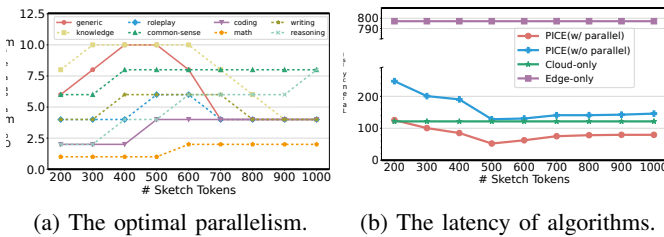


图 7: 并行生成的影响。

如图7(b)所示, 并行机制至少将PICE系统的推理延迟降低了62秒。当草图令牌数少于500时, 增加草图长度可以增强边缘推理的并行性, 从而显著降低延迟。然而, 一旦令牌数超过500个, PICE的延迟降低效果就不那么明显了。这是因为所需的KV缓存对于

提示增大, 而边缘设备的有限内存无法支持进一步增加并行度。这些结果强调了根据任务类型和草图调整并行度以优化推理效率的必要性。

通过集成学习提升质量。如图8所示, 不同问题类别下SLM的置信度排名差异显著, 凸显了每个模型的专业能力。利用这种多样性, PICE采用集成学习方法来选择最佳答案。如图9所示, 该方法在几乎所有类别中提高了回答质量, 除了编码和反事实情境, 其中角色扮演(13.95%)和知识(11.69%)类别的提升最为显著。

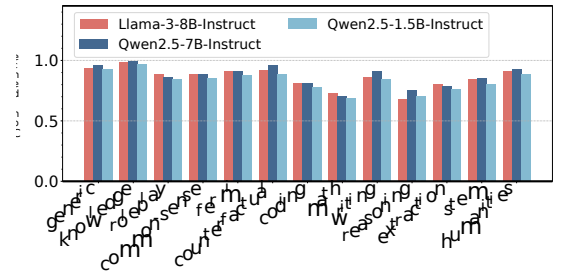


图 8: SLM 的置信度。

此外, 整体响应质量提高了2.8%, 这表明集成学习在结合各个模型的优势方面是有效的。在角色扮演和知识等类别中的显著提升凸显了集成模型在更好地捕捉细腻和详细信息方面的能力, 从而确保在需要多样化模型能力的任务中输出更高质量的结果。

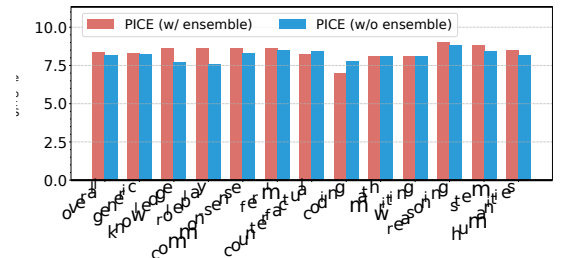


图9: 集成学习对响应质量的影响。

通过模型微调提高性能。Th

experiments demonstrate the impact of the fine-tuning component on the model’s ability to generate concise and semantically complete sketches. As shown in Fig. 10, compared to the base model, the fine-tuned model achieves a reduction in sketch length in several categories. For instance, the writing category decreases significantly from 52.3 to 42.6, and knowledge from 36.9 to 27.7. In contrast, slight increases in categories like counterfactual (from 25 to 26.4) and generic (from 18.7 to 20.7) suggest that the fine-tuning process preserves or even enhances performance in these specific areas, likely by generating longer sketches to express semantics.

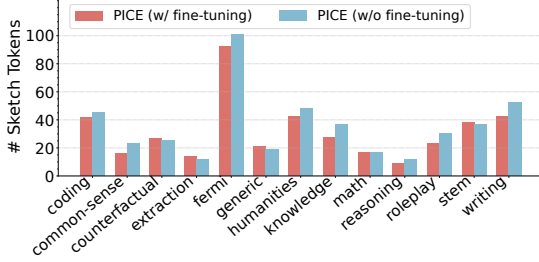


Fig. 10: The length of sketches across categories.

The results shown in Fig. 11 illustrate the model’s response quality before and after applying reinforcement learning from AI feedback. The results demonstrate the algorithm’s effectiveness, with the fine-tuned model achieving higher scores in categories like generic, math, and reasoning, indicating improved response quality. However, in knowledge and writing, the fine-tuned model performs slightly worse, likely due to its focus on generating concise sketches, which can occasionally sacrifice critical semantic details. This highlights a trade-off between conciseness and completeness, especially for tasks needing detailed explanations.

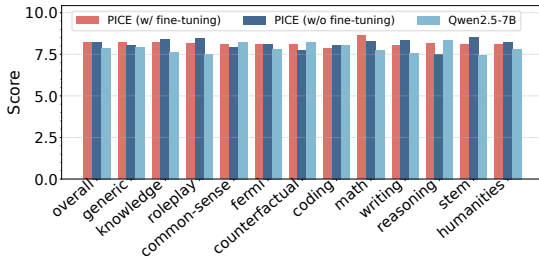


Fig. 11: Impact of the fine-tuning component on response quality across categories.

D. Sensitivity Analysis

We conduct experiments under varying conditions, including different requests per minute (RPM), job queue lengths, and network environments, to comprehensively evaluate the robustness and adaptability of PICE.

Impact of RPM. Fig. 12 illustrates the effect of varying RPM on system performance. When the RPM is below 20, which corresponds to the maximum batch size supported by the LLM in the cloud, the performance of PICE is comparable to the Cloud-only approach. However, as the RPM exceeds 20,

PICE continues to enhance throughput by offloading tasks to edge devices, whereas the throughput of Cloud-only remains static and its end-to-end latency increases sharply due to longer wait times. While the Routing method attempts to dynamically allocate requests to edge devices, its efficiency is limited by the constrained resources available at the edge.

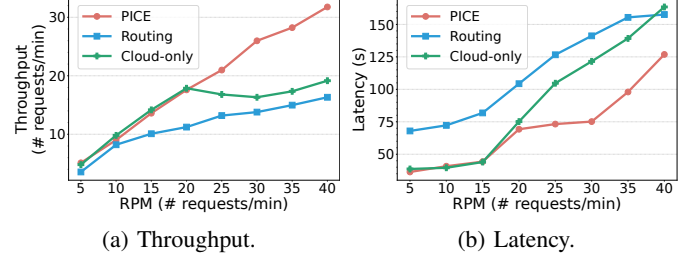


Fig. 12: The impact of RPM (requests per minute).

Impact of the job queue. The configuration of the job queue significantly affects PICE’s scheduling strategy. As shown in Fig. 13, optimal throughput is achieved when the queue length is set to four, allowing each edge device to handle one pending request while the cloud-based LLM manages the remaining workload. When the queue length exceeds eight, waiting times increase significantly, leading to a substantial rise in overall latency. This highlights the importance of aligning job queue parameters with the number of edge devices and the latency associated with edge inference. Proper tuning of these parameters ensures efficient resource utilization and minimizes delays, enhancing the overall system performance.

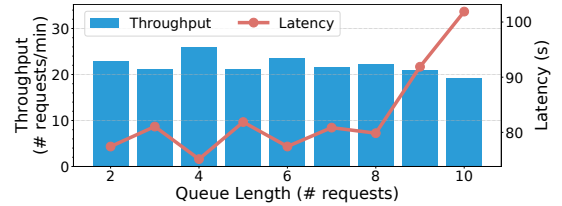


Fig. 13: The impact of job queue length.

Impact of bandwidth. Fig. 14 illustrates the effects of varying bandwidth between cloud and edge devices on the latency and throughput of the LLM inference system. In Fig. 14, throughput is analyzed under different bandwidth conditions. The PICE approach demonstrates consistently higher throughput compared to both Cloud-only and Routing methods across all bandwidth levels. This indicates the efficiency of PICE in leveraging edge resources to offload tasks and maintain performance. The Cloud-only method shows limited improvement with increasing bandwidth, highlighting its reliance on centralized resources. Meanwhile, the Routing approach underperforms due to suboptimal task allocation.

Fig. 14 shows that PICE achieves the lowest latency across all bandwidths due to dynamic scheduling and task offloading. Across all methods, including PICE, Cloud-only, and Routing, latency shows minimal variation as bandwidth increases. This is because the primary computational overhead lies in inference, while data transmission between the cloud and edge devices—limited to queries or sketches—takes only a

实验表明，微调组件对模型生成简洁且语义完整的草图的能力具有影响。如图10所示，与基础模型相比，微调模型在多个类别中实现了草图长度的缩短。例如，写作类别显著从52.3降至42.6，知识类别从36.9降至27.7。相反，诸如反事实（从25增加到26.4）和通用类别（从18.7增加到20.7）的轻微增长表明，微调过程在这些特定领域保持甚至提升了性能，这可能是通过生成更长的草图来表达语义实现的。

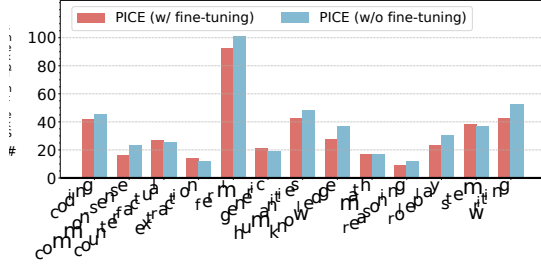


图10：各类别草图的长度。

图11所示的结果展示了在应用基于AI反馈的强化学习前后，模型的响应质量。结果表明该算法的有效性，经过微调的模型在通用、数学和推理等类别中获得了更高的评分，表明响应质量有所提高。然而，在知识和写作方面，微调后的模型表现略差，这很可能是由于其专注于生成简洁概述，偶尔会牺牲关键的语义细节。这突出了简洁性与完整性之间的权衡，尤其对于需要详细解释的任务。

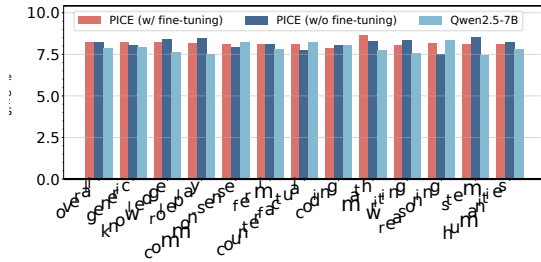


图 11：微调组件对各类别响应质量的影响。

D. Sensitivity Analysis

我们在不同条件下进行实验，包括不同的每分钟请求数（RPM）、作业队列长度和网络环境，以全面评估 PICE 的鲁棒性和适应性。

RPM的影响。图12说明了不同RPM对系统性能的影响。当RPM低于20时，这对应于云中LLM支持的最大批量大小，PICE的性能与仅云方法相当。然而，当RPM超过20时，

PICE 通过将任务卸载到边缘设备来持续提高吞吐量，而仅云端的吞吐量保持不变，且由于等待时间更长，其端到端延迟急剧增加。虽然路由方法试图将请求动态分配到边缘设备，但其效率受限于边缘可用资源的限制。

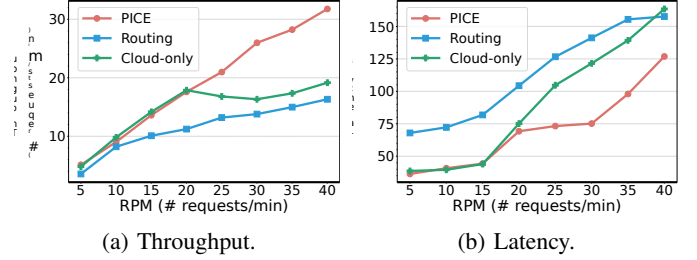


图12：RPM（每分钟请求数）的影响。

作业队列的影响。作业队列的配置显著影响 PICE 的调度策略。如图 13 所示，当队列长度设置为四时，可以实现最佳吞吐量，这允许每个边缘设备处理一个待处理请求，而基于云的 LLM 处理剩余工作负载。当队列长度超过八时，等待时间显著增加，从而导致整体延迟大幅上升。这凸显了将作业队列参数与边缘设备数量及边缘推理延迟相匹配的重要性。对这些参数进行适当调整，可以确保资源的高效利用并最小化延迟，从而提升整体系统性能。

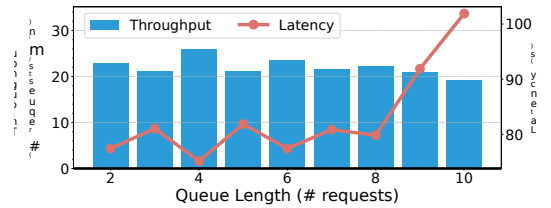


图13：作业队列长度的影响。

带宽的影响。图14展示了云端与边缘设备之间不同带宽对 LLM推理系统的延迟和吞吐量的影响。在图14中，在不同带宽条件下分析了吞吐量。PICE方法在所有带宽水平下的吞吐量始终高于仅云端和路由方法。这表明PICE在利用边缘资源卸载任务并维持性能方面的高效性。仅云端方法随着带宽增加改进有限，凸显了其依赖集中资源的特性。与此同时，路由方法由于任务分配不优化而表现不佳。

图14显示，由于动态调度和任务卸载，PICE在所有带宽下都实现了最低的延迟。在所有方法中，包括PICE、仅云以及路由，延迟随着带宽增加的变化都很小。这是因为主要的计算开销在于推理，而云与边缘设备之间的数据传输——仅限于查询或草图——仅需很少的时间。

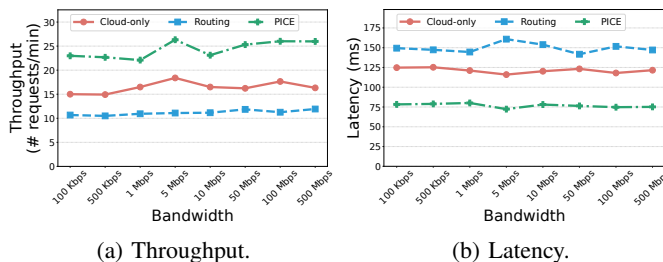


Fig. 14: The impact of bandwidth on inference efficiency.

few tens of milliseconds, even at lower bandwidths. These results highlight that network bandwidth has minimal impact on overall system performance, with inference time being the dominant factor.

VI. RELATED WORK

We review research on LLMs from the areas of response quality, inference efficiency, and the balance between the two. **Response quality.** Ning *et al.* propose SoT [6], guiding LLMs to first generate the skeleton of the answer and then filling in the content through parallel API calls. However, the applicability of SoT is constrained by its parallel mechanism. When queries involve a progressive relationship between points, such as creating a travel guide detailing Paris attractions and food along the way, SoT may encounter limitations in providing valid answers. Additionally, ensemble models have been shown to significantly enhance response quality in multi-task learning and the processing of complex data. For instance, [37] introduced an approach that integrates instruction tuning with sparse MoE techniques, effectively augmenting the expressive capacity of language models while concurrently improving both accuracy and computational efficiency. Inspired by these works, we use LLM to generate semantically complete short sentences and then use SLM to enrich each sentence, alleviating the limitations of the single model thinking paradigm.

Inference efficiency. At the system level, the speculative decoding method [14], [18], [38] adopts an SLM to generate some consecutive tokens in sequence and applies the target LLM to verify them in parallel to break the sequential decoding of LLM. Kwon *et al.* propose a novel memory management mechanism called PagedAttention and establish a high throughput system vLLM [19]. There are some other works that explore the distributed deployment of LLMs to enhance efficiency, such as EdgeShard [7] and PrivateLoRA [39]. Specifically, [7] proposes the EdgeShard framework, which partitions large language models (LLMs) into multiple segments and leverages dynamic programming algorithms to optimize their deployment across distributed devices, thereby enhancing the efficiency of LLM inference. At the model level, methodologies such as model quantization [40]–[43], pruning [44], and distillation [45] are employed to achieve model lightweighting. These approaches aim to reduce the model’s parameter size and computational complexity while preserving its performance as much as possible, thereby facilitating deployment in resource-constrained environments. [46] introduces DeepSpeed-MoE. By incorporating a MoE architecture

design and utilizing model compression techniques, such as knowledge distillation and slicing, the system reduces the size of the MoE model by a factor of 3.7.

The balance between the efficiency and quality. Tabi integrates SLMs with the LLMs to reduce inference latency while maintaining high accuracy for classification tasks [17]. For generative tasks, a hybrid inference strategy is proposed, which uses routers to dynamically assign queries to small models at the edge or large models in the cloud according to the predicted query difficulty [8], [9], [47]. DeJaVu uses the input of each layer of to predict context sparsity, and adopts asynchronous and hardware-aware methods to speed up inference while ensuring model quality [48].

VII. CONCLUSION

We propose PICE, a progressive inference system for LLM serving over cloud and edge, to effectively balance efficiency and response quality. We ensure the robust performance of PICE through a dynamic scheduler that leverages edge computing to optimize throughput, an ensemble learning mechanism that aggregates outputs from multiple small models to enhance quality, an execution optimizer that employs semantic-level parallelism to accelerate edge inference and a model fine-tuning strategy that optimizes LLMs for generating concise and semantically complete sketches. Experiments demonstrate that PICE achieves a $1.5 - 2\times$ increase in throughput and a 43% reduction in latency without compromising response quality, providing a viable solution for the widespread deployment of LLM applications. PICE is an initial attempt at semantic-centric optimization for inference efficiency, and showcases the potential of eliciting high-quality answers by explicitly planning the answer structure.

REFERENCES

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [3] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, “Qwen technical report,” *arXiv preprint arXiv:2309.16609*, 2023.
- [4] Anthropic, “Introducing claude,” <https://www.anthropic.com/index/introducing-claude>, May 2023.
- [5] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez *et al.*, “Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality,” *See https://vicuna.lmsys.org (accessed 14 April 2023)*, vol. 2, no. 3, p. 6, 2023.
- [6] X. Ning, Z. Lin, Z. Zhou, Z. Wang, H. Yang, and Y. Wang, “Skeleton-of-thought: Prompting llms for efficient parallel generation,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [7] M. Zhang, J. Cao, X. Shen, and Z. Cui, “Edgeshard: Efficient llm inference via collaborative edge computing,” *arXiv preprint arXiv:2405.14371*, 2024.
- [8] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Rühle, L. V. Lakshmanan, and A. H. Awadallah, “Hybrid llm: Cost-efficient and quality-aware query routing,” in *ICLR 2024*, 2024.
- [9] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kados, and I. Stoica, “Routellm: Learning to route llms with preference data,” *arXiv preprint arXiv:2406.18665*, 2024.
- [10] X. Zhang, Z. Huang, E. O. Taga, C. Joe-Wong, S. Oymak, and J. Chen, “Treacle: Thrifty reasoning via context-aware llm and prompt selection,” *arXiv preprint arXiv:2404.13082*, 2024.

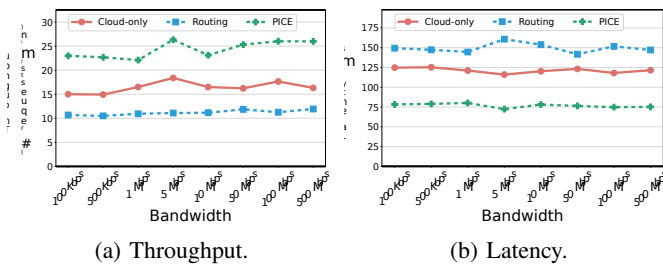


图14: 带宽对推理效率的影响。

即使在较低带宽下,也只有几十毫秒。这些结果表明,网络带宽对整体系统性能的影响很小,推理时间是主要因素。

六、相关工作

我们回顾了关于大语言模型 (LLMs) 的研究,涉及响应质量、推理效率以及两者之间的平衡。响应质量。Ning *et al.* 提出了SoT [6], 指导LLMs先生成答案的骨架,然后通过并行API调用填充内容。然而,SoT的适用性受其并行机制的限制。当查询涉及点与点之间的渐进关系时,例如创建详细描述巴黎景点和沿途美食的旅行指南,SoT在提供有效答案方面可能会遇到局限性。此外,研究表明,在多任务学习和复杂数据处理过程中,集成模型显著提升了响应质量。例如,[37]提出了一种将指令调优与稀疏MoE技术相结合的方法,有效增强了语言模型的表达能力,同时提升了准确性和计算效率。受这些工作的启发,我们使用LLM生成

推理效率。在系统层面,推测解码方法 [14]、[18]、[38] 采用 SLM 来生成一些连续的序列 token, 并并行应用目标 LLM 来验证它们, 以打破 LLM 的顺序解码。Kwon *et al.* 提出了一种称为 PagedAttention 的新型内存管理机制, 并建立了高吞吐量系统 vLLM [19]。还有一些其他工作探索了 LLM 的分布式部署以提高效率, 例如 EdgeShard [7] 和 PrivateLoRA [39]。具体来说, [7] 提出了 EdgeShard 框架, 将大型语言模型 (LLM) 划分为多个段, 并利用动态规划算法优化它们在分布式设备上的部署, 从而提升 LLM 推理的效率。在模型层面, 采用了模型量化 [40]-[43]、剪枝 [44] 和蒸馏 [45] 等方法来实现模型轻量化。这些方法旨在减少模型的参数

通过设计和使用模型压缩技术, 例如知识蒸馏和切片, 该系统将 MoE 模型的大小缩小了 3.7 倍。

效率与质量之间的平衡。Tabi 将 SLM 与 LLM 集成, 以在保持分类任务高准确率的同时减少推理延迟 [17]。对于生成任务, 提出了一种混合推理策略, 该策略使用路由器根据预测的查询难度动态地将查询分配到边缘的小模型或云中的大模型 [8], [9], [47]。DejaVu 使用每一层的输入来预测上下文稀疏性, 并采用异步和硬件感知的方法来加速推理, 同时确保模型质量 [48]。

七、结论

我们提出了 PICE, 这是一种用于云端和边缘的大型语言模型 (LLM) 服务的渐进式推理系统, 旨在有效平衡效率和响应质量。我们通过动态调度器确保 PICE 的稳健性能, 该调度器利用边缘计算优化吞吐量; 通过集成学习机制聚合多个小模型的输出以提升质量; 通过执行优化器利用语义层面的并行加速边缘推理; 以及通过模型微调策略优化 LLM, 使其生成简明且语义完整的草图。实验证明, PICE 在不影响响应质量的情况下, 实现了吞吐量提高 1.5 – 2× 和延迟降低 43%, 为 LLM 应用的广泛部署提供了可行的解决方案。PICE 是面向语义的推理效率优化的初步尝试, 并展示了通过显式规划答案结构来引出高质量答案的潜力。

参考文献

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 技术报告”, *arXiv preprint arXiv:2303.08774*, 2023.
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: 开源基础模型及微调聊天模型”, *arXiv preprint arXiv:2307.09288*, 2023.
- [3] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, “Qwen 技术报告”, *arXiv preprint arXiv:2309.16609*, 2023.
- [4] Anthropic, “介绍 Claude”, <https://www.anthropic.com/index/introducing-claude>, 2023 年5月.
- [5] W.-L. Chiang, Z. Li, Z. Lin, Y. Sheng, Z. Wu, H. Zhang, L. Zheng, S. Zhuang, Y. Zhuang, J. E. Gonzalez *et al.*, “Vicuna: 一款以 90%* ChatGPT 质量令人印象深刻的开源聊天机器人”, *See https://vicuna.lmsys.org (accessed 14 April 2023)*, 第 2 卷, 第 3 期, 第 6 页, 2023.
- [6] X. Ning, Z. Lin, Z. Zhou, Z. Wang, H. Yang, 和 Y. Wang, “思维骨架: 提示大型语言模型进行高效并行生成”, 发表于 *The Twelfth International Conference on Learning Representations*, 2024.
- [7] M. Z

- [10] X. Zhang, Z. Huang, E. O. Taga, C. Joe-Wong, S. Oymak, 和 J. Chen, “Treacle: 通过上下文感知的大语言模型和提示选择进行节俭推理”, *arXiv preprint arXiv:2404.13082*, 2024.

- [11] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2022)*, 2022, pp. 521–538.
- [12] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring massive multitask language understanding," *arXiv preprint arXiv:2009.03300*, 2020.
- [13] H. Hua, Y. Li, T. Wang, N. Dong, W. Li, and J. Cao, "Edge computing with artificial intelligence: A machine learning perspective," *ACM Computing Surveys*, vol. 55, no. 9, pp. 1–35, 2023.
- [14] D. Xu, W. Yin, X. Jin, Y. Zhang, S. Wei, M. Xu, and X. Liu, "Llmcd: Fast and scalable on-device large language model inference," *arXiv preprint arXiv:2309.04255*, 2023.
- [15] K. Srivatsa, K. K. Maurya, and E. Kochmar, "Harnessing the power of multiple minds: Lessons learned from llm routing," *arXiv preprint arXiv:2405.00467*, 2024.
- [16] B. Lin, T. Peng, C. Zhang, M. Sun, L. Li, H. Zhao, W. Xiao, Q. Xu, X. Qiu, S. Li *et al.*, "Infinite-llm: Efficient llm service for long context with distillation and distributed kv-cache," *arXiv preprint arXiv:2401.02669*, 2024.
- [17] Y. Wang, K. Chen, H. Tan, and K. Guo, "Tabi: An efficient multi-level inference system for large language models," in *EuroSys 2023*, 2023.
- [18] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning (ICML 2023)*. PMLR, 2023, pp. 19 274–19 286.
- [19] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *SOSP 2023*, 2023.
- [20] Y. Yang, Y. Xu, and L. Jiao, "A queueing theoretic perspective on low-latency llm inference with variable token length," *arXiv preprint arXiv:2407.05347*, 2024.
- [21] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "Quest: Query-aware sparsity for efficient long-context llm inference," *arXiv preprint arXiv:2406.10774*, 2024.
- [22] Z. Zheng, X. Ren, F. Xue, Y. Luo, X. Jiang, and Y. You, "Response length perception and sequence scheduling: An llm-empowered llm inference pipeline," *NeurIPS 2023*, vol. 36, 2024.
- [23] S. Laskaridis, S. I. Venieris, M. Almeida, I. Leontiadis, and N. D. Lane, "Spinn: synergistic progressive inference of neural networks over device and cloud," in *Proceedings of the 26th annual international conference on mobile computing and networking (MobiCom 2020)*, 2020, pp. 1–15.
- [24] R. T. Marler and J. S. Arora, "Survey of multi-objective optimization methods for engineering," *Structural and multidisciplinary optimization*, vol. 26, pp. 369–395, 2004.
- [25] J. Salazar, D. Liang, T. Q. Nguyen, and K. Kirchhoff, "Masked language model scoring," in *Proceedings of the ACL 2020*, 2020.
- [26] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "Bertscore: Evaluating text generation with bert," in *International Conference on Learning Representations*, 2019.
- [27] D. Jiang, X. Ren, and B. Y. Lin, "Llm-blender: Ensembling large language models with pairwise ranking and generative fusion," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL 2023)*, 2023, pp. 14 165–14 178.
- [28] F. Jelinek, R. L. Mercer, L. R. Bahl, and J. K. Baker, "Perplexity—a measure of the difficulty of speech recognition tasks," *The Journal of the Acoustical Society of America*, vol. 62, no. S1, pp. S63–S63, 1977.
- [29] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in *Text summarization branches out*, 2004, pp. 74–81.
- [30] S. Kullback and R. A. Leibler, "On information and sufficiency," *The annals of mathematical statistics*, vol. 22, no. 1, pp. 79–86, 1951.
- [31] M. Geist, B. Scherrer, and O. Pietquin, "A theory of regularized markov decision processes," in *International Conference on Machine Learning*. PMLR, 2019, pp. 2160–2169.
- [32] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in ai safety," *arXiv preprint arXiv:1606.06565*, 2016.
- [33] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, "Pytorch: An imperative style, high-performance deep learning library," *Advances in neural information processing systems*, vol. 32, 2019.
- [34] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, 2020, pp. 38–45.
- [35] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing *et al.*, "Judging llm-as-a-judge with mt-bench and chatbot arena," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [36] J. C. Zhihong Chen, F. J. Hongbo Zhang, F. Y. Guiming Chen, J. L. Tiannan Wang, Z. Z. Chen Zhang, X. W. Jianquan Li, and B. W. Haizhou Li, "Llm zoo: democratizing chatgpt," <https://github.com/FreedomIntelligence/LLMZoo>, May 2023.
- [37] S. Shen, L. Hou, Y. Zhou, N. Du, S. Longpre, J. Wei, H. W. Chung, B. Zoph, W. Fedus, X. Chen *et al.*, "Mixture-of-experts meets instruction tuning: A winning combination for large language models," *arXiv preprint arXiv:2305.14705*, 2023.
- [38] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi *et al.*, "Specinfer: Accelerating large language model serving with tree-based speculative inference and verification," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, (ASPLOS 2024), 2024, pp. 932–949.
- [39] Y. Wang, Y. Lin, X. Zeng, and G. Zhang, "Privatelora for efficient privacy preserving llm," *arXiv preprint arXiv:2311.14030*, 2023.
- [40] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, "Squeezellm: Dense-and-sparse quantization," *arXiv preprint arXiv:2306.07629*, 2023.
- [41] W. Huang, X. Ma, H. Qin, X. Zheng, C. Lv, H. Chen, J. Luo, X. Qi, X. Liu, and M. Magno, "How good are low-bit quantized llama3 models? an empirical study," *arXiv preprint arXiv:2404.14047*, 2024.
- [42] Y. Xu, L. Xie, X. Gu, X. Chen, H. Chang, H. Zhang, Z. Chen, X. Zhang, and Q. Tian, "Qa-lora: Quantization-aware low-rank adaptation of large language models," *arXiv preprint arXiv:2309.14717*, 2023.
- [43] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, "Kivi: A tuning-free asymmetric 2bit quantization for kv cache," *arXiv preprint arXiv:2402.02750*, 2024.
- [44] S. Guo, J. Xu, L. L. Zhang, and M. Yang, "Compresso: Structured pruning with collaborative prompting learns compact large language models," *arXiv preprint arXiv:2310.05015*, 2023.
- [45] Z. Yang, L. Shou, M. Gong, W. Lin, and D. Jiang, "Model compression with two-stage multi-teacher knowledge distillation for web question answering system," in *Proceedings of the 13th International Conference on Web Search and Data Mining*, 2020, pp. 690–698.
- [46] S. Rajbhandari, C. Li, Z. Yao, M. Zhang, R. Y. Aminabadi, A. A. Awan, J. Rasley, and Y. He, "DeepSpeed-moe: Advancing mixture-of-experts inference and training to power next-generation ai scale," in *International conference on machine learning*. PMLR, 2022, pp. 18 332–18 346.
- [47] Y. Xu, X. Kong, T. Chen, and D. Zhuo, "Conveyor: Efficient tool-aware llm serving with tool partial execution," *arXiv preprint arXiv:2406.00059*, 2024.
- [48] Z. Liu, J. Wang, T. Dao, T. Zhou, B. Yuan, Z. Song, A. Shrivastava, C. Zhang, Y. Tian, C. Re *et al.*, "Deja vu: Contextual sparsity for efficient llms at inference time," in *International Conference on Machine Learning*. PMLR, 2023, pp. 22 137–22 176.

- [11] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim 和 B.-G. Chun, “Orca: 用于基于Transformer生成模型的分布式服务系统”, 发表于 *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2022)*, 2022, 第 521–538 页。[12] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song 和 J. Steinhardt, “测量大规模多任务语言理解”, *arXiv preprint arXiv:2009.03300*, 2020。[13] H. Hua, Y. Li, T. Wang, N. Dong, W. Li 和 J. Cao, “人工智能的边缘计算: 机器学习视角”, *ACM Computing Surveys*, 第 55 卷, 第 9 期, 第 1–35 页, 2023。[14] D. Xu, W. Yin, X. Jin, Y. Zhang, S. Wei, M. Xu 和 X. Liu, “Llmcad: 快速且可扩展的设备端大型语言模型推理”, *arXiv preprint arXiv:2309.04255*, 2023。[15] K. Srivatsa, K. K. Maurya 和 E. Kochmar, “利用多智能体的力量: 从LLM路由中学到的经验”, *arXiv preprint arXiv:2405.00467*, 2024。[16] B. Lin, T. Peng, C. Zhang, M. Sun, L. Li, H. Zhao, W. Xiao, Q. Xu, X. Qiu, S. Li *et al.*, “Infinite-LLM: 用于长上下文的高效LLM服务, 结合DistAttention和分布式KV缓存”, *arXiv preprint*
- [35] 郑立、蒋伟龙、盛毅、庄珊、吴震、庄宇、林哲、李哲、李东、邢峥 *et al.*, “用 MT-Bench 和 Chatbot Arena 评判 LLM 作为裁判”, *Advances in Neural Information Processing Systems*, 第36卷, 2024。[36] 陈志宏、张鸿波、陈贵明、王天南、张振、李建权、李海洲, “LLM Zoo: 让 ChatGPT 民主化”, <https://github.com/FreedomIntelligence/LLMZoo>, 2023年5月。[37] 沈思、侯亮、周毅、杜楠、Longpre S、魏俊、钟宏伟、Zoph B、Fedus W、陈翔 *et al.*, “专家混合遇上指令调优: 大语言模型的获胜组合”, *arXiv preprint arXiv:2305.14705*, 2023。[38] 缪翔、Oliaro G、张震、程晓、王哲、张哲、Wong R. Y. Y.、朱安、杨立、石翔 *et al.*, “SpecInfer: 通过基于树的推测推理与验证加速大语言模型服务”, 发表于 *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, (ASPLOS 2024)*, 2024, 页932–949。[39] 王阳、林颖、曾曦、张刚, “PrivateLora 用于高效的隐私保护 LLM